Instituto Politécnico do Cávado e do Ave
Escola Técnica Superior Profissional

Curso Técnico Superior em APLICAÇÕES MÓVEIS

Estágio Curricular

Guilherme Fonte Peixoto – 31915

Curso Técnico Superior Profissional em Aplicações Móveis

Braga | março, 2026



Instituto Politécnico do Cávado e do Ave
Escola Técnica Superior Profissional

Curso Técnico Superior em APLICAÇÕES MÓVEIS

Relatório de Estágio / Projeto

Estágio realizado na empresa:

Edigma

Orientador do IPCA:

Roberto António Gonçalves Pereira

Supervisor da Entidade de Acolhimento:

Márcio Dias

Março de 2026

Agradecimentos

Quero começar por agradecer à Edigma, a empresa de acolhimento deste estágio curricular, pela forma como me recebeu e por todo o acolhimento e os ensinamentos que me proporcionou ao longo destes meses. Foi na Edigma que tive oportunidade de crescer tecnicamente e de compreender, na prática, o que significa desenvolver software num contexto profissional real.

Um agradecimento especial aos dois Diogos, Diogo Guimarães e Diogo Puga, e ao André, pela disponibilidade e pelo apoio que me deram ao longo de todo o estágio.

Por fim, o meu maior agradecimento vai para o Márcio Dias, meu supervisor na Edigma, que foi muito mais do que um supervisor — foi um verdadeiro mentor. Acompanhou-me de perto nas fases mais críticas de arquitetura do projeto Museum Mate, sempre disponível para dar sugestões, dicas e propor melhorias, contribuindo de forma decisiva para o resultado final deste trabalho.

Resumo

O presente relatório documenta o estágio curricular realizado no âmbito do Curso Técnico Superior Profissional em Aplicações Móveis do Instituto Politécnico do Cávado e do Ave (IPCA), com início em março de 2026, na empresa Edigma, sob orientação do supervisor Márcio Dias e do orientador do IPCA, Roberto Pereira.

Numa fase inicial, o estágio focou-se na aprendizagem da framework Qt e da linguagem QML, consolidada através do desenvolvimento de três mini-projetos — um jogo da memória, um jogo da forca e um jogo de Air Hockey —, que permitiram explorar conceitos de interface gráfica, deteção de colisões, gestão de estados e lógica de interação, preparando o terreno para um projeto de maior dimensão.

O restante período de estágio foi dedicado ao desenvolvimento do Museum Mate, uma aplicação móvel concebida para funcionar como assistente digital de visita em museus e outros espaços culturais, seguindo um modelo Bring Your Own Device (BYOD) que dispensa a aquisição de hardware dedicado por parte das instituições. A solução foi estruturada numa arquitetura de quatro componentes: a aplicação móvel, desenvolvida em Qt/QML; um sistema de gestão de conteúdos Headless CMS assente no Strapi, onde toda a informação — Entidade, mapas, áreas, itens, visitas guiadas e traduções — é administrada de forma centralizada e multilingue; um painel de análise estatística, também integrado no Strapi, que regista e visualiza a utilização da aplicação; e uma infraestrutura contentorizada em Docker, responsável por garantir a distribuição e a instalação consistente do sistema em qualquer ambiente.

A partir de determinada fase do estágio, o desenvolvimento passou a seguir uma metodologia orientada por especificações (Spec-Driven Development), com o apoio do assistente de programação Claude Code, o que permitiu manter as decisões tomadas devidamente documentadas e rastreáveis ao longo de um projeto desenvolvido individualmente. Em suma, o estágio permitiu consolidar competências de programação em Qt/QML, adquirir experiência em arquiteturas backend headless, contentorização e integração de inteligência artificial, e compreender, na prática, o ciclo completo de conceção, planeamento, implementação e documentação de um produto de software real.

Índice

1. Introdução 9
1.1 Apresentação do Estágio 10
1.1.1Planeamento do estágio 11
1.2 Apresentação da empresa 12
1.2.1 Soluções tecnológicas desenvolvidas 13
1.2.2 Superfícies interativas e experiências digitais 14
2. Contexto 14
2.1 Desenvolvimento de aplicações interativas 15
2.2 Framework Qt 16
2.3 Linguagem QML 16
3. Descrição Técnica 17
3.1 Preparação e tarefas iniciais 17
3.2 Mini-Projetos 18
3.2.1 Jogo da memória 18
3.2.2 Jogo da forca 20
3.2.3 Jogo Air Hockey 22
3.3 Projeto Museum Mate 26
3.3.1. Introdução ao projeto 26
3.3.2 Problema Identificado 27
3.3.3. Objetivo da aplicação 28
3.3.4 Requisitos do Sistema 28
3.3.5 Arquitetura Geral do Sistema 31
3.3.6 Planeamento da Solução 32
3.3.7 Arquitetura Headless CMS com Strapi 34
3.3.8. Contentorização e Infraestrutura com Docker 40
3.3.9 Metodologia de Desenvolvimento: Spec-Driven Development 45
4 Desenvolvimento do Frontend 47
4.1.1 Arquitetura de Comunicação: Estratégia de Endpoint Único 47
4.1.2 Gestão de Dados, Cache e API (ApiHandler.js) 48
4.1.3 Mecanismo de Persistência e Cache Local (*Offline-First*) 49
4.1.4 Resolução Dinâmica de Recursos Multimédia 51

4.2 Módulo de Telemetria e Monitorização (AnalyticsHandler.js) 52
4.2.1 Fluxo de Processamento e Resiliência de Dados 52
4.3 Janela Principal e Controlo de Estado (main.qml) 53
4.3.1 Carregamento e Descarregamento Centralizado de Dados 53
4.3.2 Gestão da Sessão e Eventos Analíticos 54
4.3.3 Navegação com o Componente Loader 55
4.4 Ecrã de Boas-Vindas (WelcomePage.qml) 56
4.5 Ecrã Principal e Mapa Interativo (HomePage.qml) 57
4.5.1 Descrição Geral da Interface 57
4.5.2 Organização por Pisos e Filtragem de Dados 59
4.5.3 Ajuste Automático do Mapa no Ecrã 60
4.5.4 Leitura de Códigos QR com a Biblioteca QZXing 62
4.5.5 Razões e Benefícios da Automação de Percurso 63
4.6 Identificação de Obras: Scanner, Código Manual e NFC 64
4.7 Lista de Visitas Guiadas (Tours.qml) 65
4.7.1 Descrição Geral e Propósito do Ecrã 65
4.8 Ecrã da Visita Guiada Ativa (TourPage.qml) 66
4.8.1 Descrição Geral e Objetivos de Conceção 66
4.8.2 Controlo Nativo e Arquitetura do Foreground Service 67
4.8.3 Identificação Inteligente e Tratamento de Itens Globais 71
4.9 Assistente Virtual (ChatIAPanel.qml) 73
4.9.1 Motivação e objetivo da funcionalidade 73
4.9.2 Integração do assistente na aplicação 73
4.9.3 Arquitetura e processamento das perguntas 75
4.9.4 Sistema de fallback e controlo da informação gerada 76
4.9.5 Suporte multilingue e adaptação das respostas 77
4.9.6 Recolha de dados e melhoria contínua dos conteúdos 77
4.9.7 Impacto da funcionalidade 78
5. Conclusão 79
6. Bibliografia 80

Índice de Ilustrações

Figura 1 - Setup Edigma 15
Figura 2 - Jogo da Memória 19
Figura 3 - Jogo da forca 21
Figura 4 - Air Hockey 23
Figura 5 - Função checkCollision() - Air Hockey 23
Figura 6 - Timer Verificação Golo - Air Hockey 24
Figura 7 - Função Reiniciar Jogo - Air Hockey 25
Figura 8 - Fluxo geral do Sistema 32
Figura 9 - Diagrama de fluxo de navegação do utilizador 33
Figura 10 - Exemplo da coleção Área com o uso i18n 39
Figura 11 - docker-compose.yml 41
Figura 12 - Dockerfile 41
Figura 13 - restore.sh 43
Figura 14 - Fluxo de comunicação entre a interface da aplicação e o servidor 48
Figura 15 - Fluxo de Dados e Cache 50
Figura 16 - Fluxo de dados Multimédia 51
Figura 17 - Fluxo lógico do sistema de estatísticas 52
Figura 18 - Página Inicial 53
Figura 19 - função carregarDados 54
Figura 20 - Loader main 55
Figura 21 - Página de boas-vindas 56
Figura 22 - HomePage 58
Figura 23 - Função updateFilteredItems 59
Figura 24 - Fluxo do timerPositionMap 60
Figura 25 - Pins na zona inferior 61
Figura 26 - Pins na zona Superior 61
Figura 27 - Fluxo QrCode 62
Figura 28 - Leitura e processamento automático do QR Code 63
Figura 29 - Listagem de Visitas Guiadas 66
Figura 30 - Fluxo da Página TourPage.qml 67
Figura 31 - Arquitetura Foreground Service 68
Figura 32 - Lógica da TourAudioService.java 69
Figura 33 - AudioBridge 70
Figura 34 - Leitura dinamica de Item do Tour 71
Figura 35 - Leitura dinâmica de Item fora do Tour 72
Figura 36 - Chat IA 74

Lista de acrónimos

API - Application Programming Interface (Interface de Programação de Aplicações)

AR - Augmented Reality (Realidade Aumentada)

ATL - Atividades de Tempos Livres

BYOD - Bring Your Own Device (Traga o Seu Próprio Dispositivo)

CMS - Content Management System (Sistema de Gestão de Conteúdos)

HTTP - HyperText Transfer Protocol (Protocolo de Transferência de Hipertexto)

i18n - Internationalization (Internacionalização)

IA - Inteligência Artificial

ID - Identifier (Identificador)

IPCA - Instituto Politécnico do Cávado e do Ave

JDK - Java Development Kit

JNI - Java Native Interface (Interface Nativa Java)

JSON - JavaScript Object Notation

LED - Light Emitting Diode (Díodo Emissor de Luz)

NDK - Native Development Kit

NFC - Near Field Communication (Comunicação por Campo de Proximidade)

QML - Qt Modeling Language

QR - Quick Response (Resposta Rápida)

RAM - Random Access Memory (Memória de Acesso Aleatório)

REST - Representational State Transfer

SDK - Software Development Kit

VR - Virtual Reality (Realidade Virtual)

1. Introdução

O estágio curricular constitui uma componente fundamental no processo de formação dos estudantes do ensino superior, permitindo a aplicação prática dos conhecimentos adquiridos ao longo do percurso académico em contexto real de trabalho. Para além de consolidar competências técnicas, o estágio proporciona também o contacto com metodologias de desenvolvimento utilizadas no mercado de trabalho, promovendo o desenvolvimento de competências profissionais, capacidade de adaptação e resolução de problemas.

No âmbito do Curso Técnico Superior Profissional em Aplicações Móveis do Instituto Politécnico do Cávado e do Ave (IPCA), foi realizado um estágio curricular na empresa Edigma, uma empresa tecnológica especializada no desenvolvimento de soluções digitais interativas. Este estágio insere-se na área do desenvolvimento de software, com particular enfoque na criação de aplicações interativas utilizando tecnologias associadas à framework Qt e à linguagem QML.

Durante o período de estágio, o trabalho desenvolvido tem como objetivo principal aprofundar conhecimentos na área do desenvolvimento de interfaces gráficas e aplicações interativas, através da realização de exercícios práticos, mini-projetos e projetos de maior dimensão. Estes projetos permitem explorar diferentes conceitos de programação, tais como manipulação de interfaces gráficas, interação com o utilizador, lógica de aplicações e integração de dados.

Para além da componente técnica, o estágio permite também compreender o processo de desenvolvimento de software em ambiente profissional, incluindo a fase de análise de requisitos, planeamento, implementação e melhoria contínua das soluções desenvolvidas.

Assim, o presente relatório pretende documentar as atividades realizadas durante o estágio, descrevendo o contexto em que o mesmo foi desenvolvido, as tecnologias utilizadas, os projetos realizados e os principais resultados obtidos ao longo do processo de aprendizagem e desenvolvimento.

1.1 Apresentação do Estágio

O estágio curricular foi realizado na empresa Edigma e teve como principal objetivo proporcionar uma experiência prática no desenvolvimento de aplicações interativas, permitindo a aplicação e consolidação dos conhecimentos adquiridos ao longo do percurso académico. Este estágio enquadra-se na área do desenvolvimento de software, com particular enfoque na criação de interfaces gráficas e aplicações interativas utilizando tecnologias associadas à framework Qt.

Durante o período de estágio foram desenvolvidas diversas atividades técnicas com o objetivo de aprofundar conhecimentos em tecnologias específicas utilizadas pela empresa, bem como compreender os processos de desenvolvimento adotados em contexto profissional. Numa fase inicial, o trabalho incidiu na familiarização com o ambiente de desenvolvimento e com as tecnologias utilizadas, incluindo o estudo da linguagem QML (Qt Modeling Language) e da framework Qt, amplamente utilizadas no desenvolvimento de interfaces gráficas e aplicações interativas.

Após esta fase inicial de aprendizagem, foram desenvolvidos vários exercícios práticos e mini-projetos com o objetivo de consolidar os conhecimentos adquiridos. Estes projetos permitiram explorar diferentes conceitos fundamentais de programação e desenvolvimento de interfaces, incluindo a gestão de componentes visuais, manipulação de eventos de interação, organização da estrutura das aplicações e implementação de lógica de funcionamento.

Concluída a fase de introdução com os mini-projetos, o restante período de estágio foi inteiramente dedicado ao desenvolvimento de um único projeto principal de grande dimensão. Este projeto central permitiu aplicar de forma integrada todas as competências adquiridas anteriormente, englobando a criação de uma aplicação interativa complexa e a sua articulação com os serviços digitais da organização.

Desta forma, o estágio permitiu não apenas o aprofundamento de competências técnicas na área do desenvolvimento de software, mas também o contacto com metodologias de trabalho utilizadas em ambiente profissional, autonomia e capacidade de adaptação a novos desafios tecnológicos.

1.1.1Planeamento do estágio

O fluxo do estágio foi o seguinte:

Fase	Período	Atividade
Integração	Dia 1 e 2	Introdução à empresa
Mini-projetos	Semana 1 - 2	Realização de jogos
Projeto principal	Restante estágio	Museum Mate

1.1.2 Reuniões de acompanhamento

Data	Participantes	Tema
05/03/2026	Estagiário Orientador Ipca Supervisor Edigma	Plano de Estágio
20/03/2026	Estagiário e Orientador Ipca	Acompanhamento, Esclarecimento e Progresso
09/04/2026	Estagiário e Orientador Ipca	Revisão Relatório
03/06/2026	Estagiário e Orientador Ipca	Revisão de documentos e Relatório de estágio
02/07/2027	Estagiário Orientador Ipca Supervisor Edigma	Revisão e analise do estágio Nota final

1.2 Apresentação da empresa

A EDIGMA é uma empresa tecnológica portuguesa sediada em Braga, especializada no desenvolvimento de soluções interativas para espaços físicos. A empresa destaca-se como uma das referências nacionais nas áreas de digital signage, gestão de filas e experiências multimédia interativas.

Ao longo da sua atividade, a empresa tem desenvolvido projetos inovadores em diversos sectores, incluindo cultura, banca, retalho e mobilidade. 2As suas soluções procuram melhorar a interação entre pessoas e tecnologia em espaços públicos e comerciais, proporcionando experiências digitais mais envolventes.

A EDIGMA tem também obtido reconhecimento internacional através da atribuição de diversos prémios e distinções, entre os quais se destacam prémios na área da museologia e inovação tecnológica. Estes reconhecimentos refletem a capacidade da empresa em combinar design, tecnologia e criatividade na criação de experiências digitais diferenciadoras.

A abordagem da empresa baseia-se na integração de hardware e software próprio, incluindo equipamentos como ecrãs digitais, quiosques multitoque e soluções de realidade virtual, juntamente com plataformas de software desenvolvidas internamente.



Logo Edigma

1.2.1 Soluções tecnológicas desenvolvidas

A EDIGMA desenvolve diversas soluções tecnológicas orientadas para a melhoria da experiência do utilizador em espaços físicos. As soluções da empresa combinam hardware interativo e software próprio, permitindo criar ambientes digitais dinâmicos em locais como museus, bancos, espaços comerciais e exposições.

De forma geral, as soluções tecnológicas da empresa podem ser agrupadas nas seguintes áreas principais:

Sinalética digital (Digital Signage)

Uma das principais soluções desenvolvidas pela empresa é a plataforma EDIGMA DS, um sistema de gestão de conteúdos (CMS – Content Management System) destinado à gestão de comunicação visual em múltiplos dispositivos digitais.

Esta plataforma permite:

- gerir conteúdos digitais de forma centralizada
- atualizar remotamente informação apresentada em ecrãs
- programar conteúdos para diferentes horários ou locais
- controlar diversos dispositivos a partir de uma única plataforma

Os conteúdos podem ser apresentados em vários tipos de equipamentos, tais como:

- ecrãs LED
- montras digitais
- videowalls
- quiosques interativos

Este tipo de solução é amplamente utilizado por empresas e instituições que pretendem melhorar a comunicação visual com clientes e visitantes.

1.2.2 Superfícies interativas e experiências digitais

Para além das soluções de comunicação digital e gestão de atendimento, a EDIGMA desenvolve também instalações tecnológicas interativas destinadas a criar experiências digitais envolventes.

Entre estas soluções destacam-se:

- Mesas multitoque interativas utilizadas em museus e exposições
- Quiosques digitais interativos para informação ao público
- Experiências de realidade virtual (VR)
- Aplicações de realidade aumentada (AR)

Estas tecnologias são frequentemente utilizadas em contextos culturais e educativos, permitindo aos visitantes interagir com conteúdos digitais de forma mais dinâmica e imersiva.

2. Contexto

O estágio realizado na EDIGMA insere-se no contexto do desenvolvimento de aplicações interativas, com foco na criação de interfaces gráficas e na interação com o utilizador. A EDIGMA, como empresa tecnológica líder em soluções digitais para espaços físicos, apresenta um ambiente rico em desafios práticos e tecnológicos, oferecendo oportunidades para o desenvolvimento de competências em programação, design de interfaces e gestão de conteúdos digitais.

Durante o período de estágio, as atividades foram desenvolvidas nas instalações da EDIGMA, num ambiente colaborativo e multidisciplinar, em contacto próximo com a equipa de desenvolvimento. A integração neste contexto permitiu acompanhar as práticas utilizadas pela empresa no desenvolvimento de soluções digitais, facilitando a aprendizagem de metodologias de trabalho, ferramentas e processos adotados em contexto profissional. A Figura X apresenta o posto de trabalho utilizado ao longo do estágio, onde decorreram as atividades de desenvolvimento da aplicação Museum Mate.

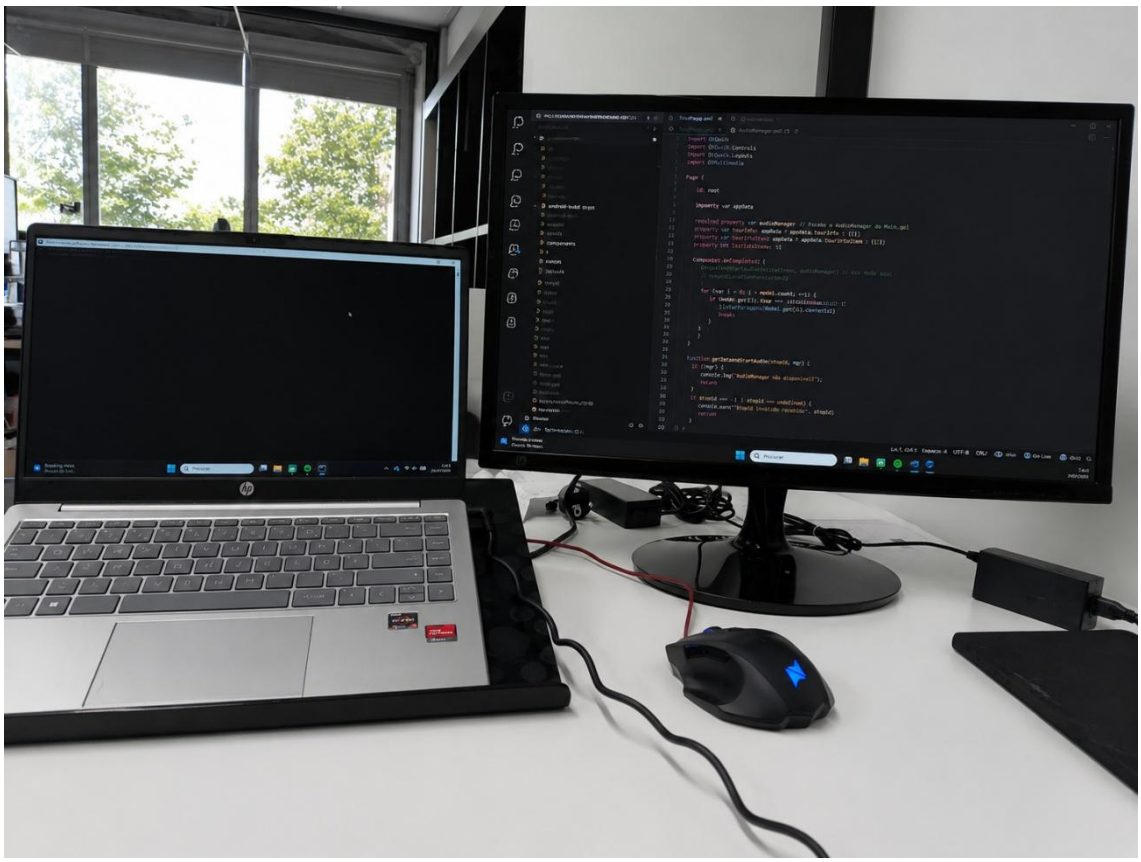


Figura 1 - Setup Edigma

2.1 Desenvolvimento de aplicações interativas

As atividades iniciais envolveram a criação de pequenos projetos e exercícios práticos com o objetivo de consolidar o conhecimento técnico.

Progressivamente, os exercícios evoluíram para mini-projetos completos, como jogos interativos (jogo da memória, jogo da forca e Air Hockey), nos quais foram aplicados conceitos de lógica de programação, colisões, controle de objetos e gestão de estados de jogo. Este percurso permitiu uma aprendizagem contínua, onde cada projeto novo introduzia desafios adicionais em termos de interatividade, lógica e usabilidade.

2.2 Framework Qt

A framework Qt constitui a base do desenvolvimento das aplicações interativas na EDIGMA. Trata-se de um framework de desenvolvimento multiplataforma que permite criar aplicações com interfaces gráficas avançadas, compatíveis com desktop, dispositivos móveis e sistemas embutidos.

O estágio incluiu a configuração de ambientes de desenvolvimento Qt, incluindo QtCreator, e a resolução de problemas relacionados com SDK, JDK, NDK e Kits para permitir a execução de aplicações em dispositivos Android. O conhecimento adquirido nesta fase será utilizado posteriormente na implementação de projetos mais complexos, incluindo aplicações para interfaces móveis e interativas.

2.3 Linguagem QML

A linguagem QML é uma linguagem declarativa utilizada para criar interfaces gráficas reativas e interativas em conjunto com Qt. Durante o estágio, foram explorados conceitos fundamentais da linguagem.

Além disso, a aprendizagem de boas práticas de programação e modularização do código foi incorporada desde os primeiros exercícios, preparando o terreno para projetos mais complexos, como o desenvolvimento de uma aplicação Museum Mate.

3. Descrição Técnica

Este capítulo apresenta o trabalho realizado durante o estágio, descrevendo as tarefas, projetos e aprendizagens técnicas que foram desenvolvidas na EDIGMA. O objetivo é documentar de forma clara o percurso realizado até ao momento, servindo como base para a análise de projetos mais complexos que serão desenvolvidos posteriormente.

O capítulo foca-se em três pontos principais: a preparação e aquisição de conhecimentos fundamentais, os mini-projetos desenvolvidos e os projetos mais complexos que constituirão a parte principal do estágio.

3.1 Preparação e tarefas iniciais

Nos primeiros dias do estágio, a prioridade foi integrar-me na equipa e compreender as tecnologias base necessárias para o desenvolvimento das aplicações. As tarefas iniciais incluíram:

- Introdução à equipa de desenvolvimento, apresentação dos membros e das áreas de atuação.
- Configuração do perfil na plataforma interna da EDIGMA, como forma de integração na equipa.
- Estudo e prática da linguagem QML, incluindo exercícios básicos de componentes gráficos, layouts e eventos interativos.

Durante este período foram abordados conceitos essenciais para o desenvolvimento de interfaces, tais como: Rectangle, Repeater, ListView, Row e Column, arrays, eventos onClicked e Flickable. Estas atividades forneceram as bases para compreender a lógica declarativa de QML e a interação entre componentes, que seriam fundamentais para os projetos seguintes.

Também foram realizados exercícios práticos de manipulação de imagens e vídeos em QML, criando arrays dinâmicos para organizar conteúdos multimédia e exibi-los em loop, assim como a preparação do ambiente QtCreator para desenvolvimento Android, resolvendo questões de SDK, JDK e NDK.

Estas tarefas iniciais permitiram consolidar os conhecimentos necessários para abordar projetos mais complexos e aplicá-los em contextos práticos.

3.2 Mini-Projetos

No decurso das semanas iniciais do estágio curricular, foram desenvolvidos três mini-projetos práticos. O desenvolvimento destas aplicações teve como propósito fundamental a consolidação prática dos conceitos teóricos da framework Qt e da linguagem QML, englobando a estruturação de interfaces dinâmicas, a gestão de estados e a implementação de lógicas de interação síncronas e assíncronas. Adicionalmente, esta fase inicial permitiu a familiarização com as metodologias de trabalho e fluxos de desenvolvimento adotados na entidade de acolhimento.

3.2.1 Jogo da memória

Objetivo do projeto

Este mini-projeto teve como objetivo desenvolver um jogo da memória em QML com diferentes níveis de dificuldade, aplicando conceitos de interface gráfica, lógica de programação e gestão de estados. Serviu também como introdução ao ambiente de desenvolvimento da empresa e à metodologia de trabalho com orientação.

Análise e estrutura do problema

Para a correta modelação do problema, identificaram-se os seguintes requisitos funcionais e técnicos obrigatórios:

- **Parametrização da Dificuldade:** Disponibilização de tabuleiros com dimensões distintas (grelhas de 2x2, 4x4 e 6x6 cartas);
- **Lógica de Correspondência:** Mecanismo de validação de pares homólogos com base em identificadores únicos;
- **Restrições Temporais:** Integração de um sistema de temporização decrescente associado a opções de desistência por parte do utilizador;
- **Responsividade:** Garantia de uma correta adaptação do layout visual a diferentes resoluções de dispositivos móveis.

Implementação e funcionamento

A arquitetura do projeto foi segmentada de forma modular, recorrendo a componentes reutilizáveis, Loaders para a alternância de ecrãs e Timers para o controlo de eventos bi-estáveis.

A lógica nuclear baseia-se na retenção temporária do estado de duas cartas selecionadas pelo utilizador. Caso se verifique equivalência entre os atributos dos objetos, o respetivo estado visual é alterado permanentemente para descoberto; em caso de divergência, invoca-se uma rotina de atraso (*delay*) que reverte o estado das cartas para oculto. A distribuição aleatória dos elementos é executada no arranque de cada sessão de jogo, mitigando padrões repetitivos.

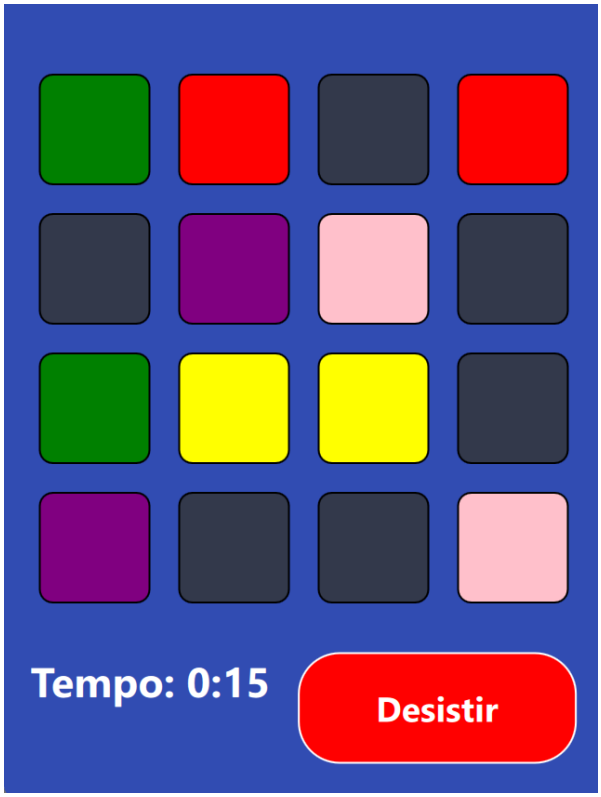


Figura 2 - Jogo da Memória

A interface resultante encontra-se ilustrada na Figura 2, onde as cartas por desvendar se apresentam a cinzento e os pares validados expõem o respetivo conteúdo cromático.

3.2.2 Jogo da forca

Objetivo do projeto

Este projeto teve como meta a implementação de uma versão digital do jogo da forca através da linguagem QML. O desafio principal consistiu no desenho de uma arquitetura de dados flexível e desacoplada, capaz de suportar a alteração dinâmica de categorias temáticas de palavras sem a necessidade de reestruturar o núcleo lógico do algoritmo. O projeto foi executado de forma autónoma, estimulando a independência técnica.

Análise do problema

Antes do desenvolvimento do jogo foi necessário analisar os elementos fundamentais para o seu funcionamento. O objetivo era criar um sistema que permitisse:

- Selecionar um tema de jogo;
- Escolher uma palavra aleatória pertencente ao tema;
- Apresentar traços correspondentes ao número de letras da palavra;
- Permitir ao utilizador selecionar letras através de um teclado virtual;
- Verificar se a letra escolhida existe na palavra;
- Mostrar progressivamente o boneco da forca sempre que ocorre um erro;
- Detetar situações de vitória ou derrota.

Foi também considerado importante garantir que o jogo fosse facilmente adaptável a diferentes conjuntos de palavras, através de uma estrutura modular.

Implementação e funcionamento

O sistema foi estruturado em três módulos principais: ecrã de configuração de modo, ecrã de seleção temática e matriz de jogo.

Após a inicialização da partida, o tratamento de inputs é efetuado através de uma matriz de botões que emula o teclado. A deteção de uma letra correta desencadeia a atualização do modelo visual, revelando o caráter na posição correspondente da palavra oculta. Inversamente, a submissão de um caráter incorreto incrementa o estado de erro, atualizando graficamente o componente visual da forca através de opacidades dinâmicas e caminhos vetoriais coordenados.

O desfecho da sessão é ditado pela verificação das condições de vitória (totalidade de caracteres descobertos) ou de derrota (limite máximo de tentativas excedido), conforme ilustrado na Figura 3.

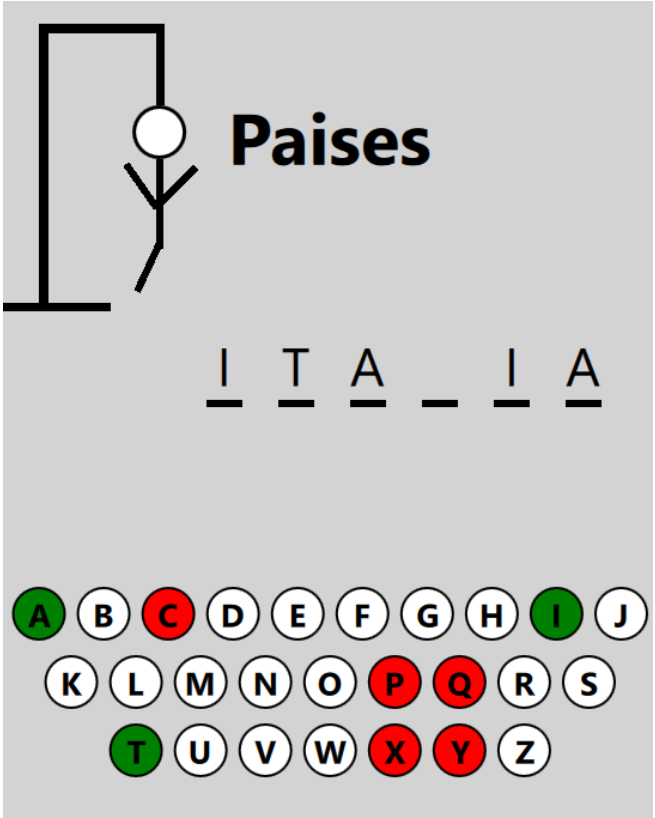


Figura 3 - Jogo da forca

3.2.3 Jogo Air Hockey

Objetivo do projeto

Este mini-projeto teve como objetivo desenvolver uma versão simplificada do jogo Air Hockey em QML, onde dois jogadores controlam discos com o objetivo de marcar golos na baliza adversária.

Este projeto foi proposto como um desafio técnico adicional, permitindo explorar novos conceitos como deteção de colisões, movimento dinâmico de objetos e controlo da direção da bola. O desenvolvimento foi realizado de forma maioritariamente autónoma, com recurso pontual à documentação oficial.

Analise do Problema

Antes de iniciar o desenvolvimento foi necessário analisar os principais elementos necessários para o funcionamento do jogo. O sistema deveria permitir:

- Controlar dois jogadores através do movimento do toque;
- Implementar uma bola que se movimenta no campo de jogo;
- Detetar colisões entre a bola e os jogadores;
- Alterar a direção da bola após colisão;
- Detetar quando a bola entra na baliza;
- Atualizar a pontuação dos jogadores.

Outro desafio importante foi garantir que cada jogador apenas pudesse mover o seu disco dentro da sua metade do campo, evitando que um jogador invadisse a área do adversário.

Também foi necessário implementar um sistema de movimentação contínua da bola para simular o comportamento típico de um jogo de Air Hockey.

Implementação e funcionamento

A aplicação estruturou-se na definição da área de jogo (limites delimitadores e balizas), na parametrização dos jogadores (controlados diretamente através de eventos de toque no ecrã) e no motor de física simplificado, visível na Figura 4.

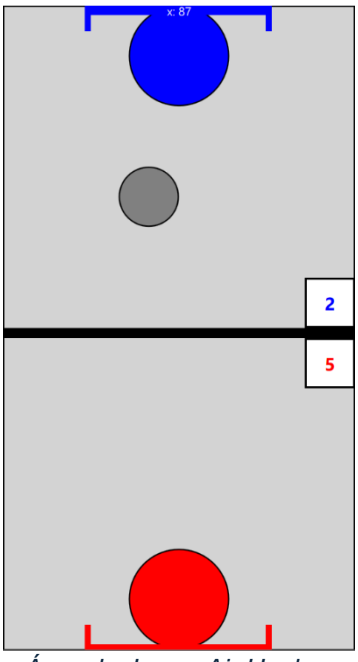


Figura 4 - Air Hockey

O funcionamento lógico e o processamento de eventos do jogo assentam em três pilares fundamentais:

- **Sistema de Deteção de Colisões:** Para viabilizar a interação física entre os corpos, foi implementada a função autónoma checkCollision() (Figura 5). Esta rotina avalia periodicamente a sobreposição geométrica das coordenadas dos objetos circulares. Aquando da intersecção, o algoritmo determina a posição relativa do impacto, discriminando se a colisão ocorreu primariamente na componente horizontal ou vertical, de forma a inverter o sinal algébrico do vetor de velocidade correspondente e simular o efeito de ricochete.

```
function  checkCollision(a,b){
    return a.x < b.x + b.width &&
           a.x + a.width > b.x &&
           a.y < b.y + b.height &&
           a.y + a.height > b.y
}
```

Figura 5 - Função checkCollision() - Air Hockey

Sistema de movimento da bola

O movimento da bola é controlado através de vários Timers, responsáveis por atualizar continuamente a sua posição no ecrã.

Estes temporizadores permitem mover a bola:

- para a esquerda ou direita;
- para cima ou para baixo.

A ativação e desativação destes timers permite alterar a direção da bola sempre que ocorre uma colisão ou quando a bola atinge os limites do campo.

Sistema de golos e pontuação

O jogo possui duas zonas de golo, uma para cada jogador. Quando a bola entra numa dessas zonas, o sistema reconhece que ocorreu um golo.

A deteção do golo é feita através da verificação de colisão entre a bola e as áreas definidas como baliza usando um Timer como esta na figura 6, que verifica a cada 70 milissegundos se foi golo. Quando um golo é marcado:

- a pontuação do jogador adversário é incrementada;
- Ativa o Timer e consequentemente a função *restartGame () da figura 7;*
- os jogadores voltam à posição inicial;
- a bola é reposicionada no campo para reiniciar a jogada.

A pontuação de cada jogador é apresentada na interface através de elementos Text, permitindo acompanhar o resultado da partida em tempo real.

```
Timer {
    id: checkGoalsTimer
    running: true
    interval: 70
    repeat: true

    onTriggered: {
        if (checkCollision(bola, recGolo)) {
            contaGolos++
            console.log("GOLO" + contaGolos)
            rec1.x = (itemTudo.width - rec1.width)/ 2
            rec1.y = 0
            rec2.x = (itemTudo.width - rec2.width)/ 2
```

Figura 6 - Timer Verificação Golo - Air Hockey

```
function restartGame () {
    if (checkCollision(bola, recGolo)) {
        pointsPlayer2 = pointsPlayer2 + 1
        checkGoalsTimer.running = true
    }
    if (checkCollision(bola, recGolo2)) {
        pointsPlayer1 = pointsPlayer1 + 1
        checkGoalsTimer.running = true
    }
}
```

Figura 7 - Função Reiniciar Jogo - Air Hockey

3.3 Projeto Museum Mate

O presente capítulo delimita a componente central e estruturante do estágio curricular desenvolvido na Edigma. No dia 12 de março de 2026, em articulação com o supervisor da entidade de acolhimento, foi delineada a conceção de um ecossistema de software escalável, idealizado para aplicar e aprofundar competências avançadas em arquitetura de sistemas, levantamento de requisitos técnicos e desenvolvimento full-stack.

O projeto compreende a implementação de uma infraestrutura de *backend* assente num sistema *Headless CMS* (Strapi) isolado em ambiente de contentorização (*Docker*), a criação de scripts de automação, bem como o desenho de uma interface móvel nativa recorrendo à framework *Qt* e à linguagem *QML*, configuradas especificamente para ambientes de execução móvel. Em detrimento de uma mera listagem cronológica das tarefas executadas, esta secção descreve com rigor a lógica de engenharia de software, as restrições de sistema e as decisões arquiteturais que sustentam a aplicação.

3.3.1. Introdução ao projeto

Após a consolidação dos conhecimentos adquiridos nas fases iniciais do estágio, foi possível evoluir para o desenvolvimento de projetos com maior dimensão e complexidade, exigindo uma abordagem mais estruturada e um planeamento mais cuidadoso.

E foi nesse sentido que surgiu a oportunidade de realizar o projeto que apelidei de Museum mate, um projeto que foi proposto pelo meu supervisor no dia 12 de Marco, um projeto mais estruturado com o objetivo de aplicar de forma integrada os conhecimentos adquiridos até ao momento. Este desafio marcou uma nova etapa, exigindo uma abordagem mais organizada, maior autonomia e uma maior atenção ao planeamento e desenvolvimento da aplicação

Foi concebida a realização de uma aplicação de assistente de visita, para melhorar a experiência do visitante em espaços culturais e turísticos, mais especificamente museus, permitindo uma exploração mais interativa, autónoma e personalizada através do próprio dispositivo móvel do visitante.

3.3.2 Problema Identificado

A oportunidade e a necessidade de conceção deste projeto emergiram da observação direta de uma lacuna tecnológica no setor da mediação cultural e da sinalética digital interativa. Embora a esmagadora maioria dos espaços museológicos disponibilize dispositivos tradicionais de áudio-guia ao público, este modelo operacional acarreta limitações severas de engenharia e logística:

- Rigidez na Atualização: Qualquer alteração ou correção nos conteúdos expositivos exige intervenções manuais complexas nos dispositivos, comprometendo a escalabilidade da informação.
- Custos de Infraestrutura: A aquisição, manutenção e higienização de hardware proprietário representam um encargo financeiro contínuo e estático para as instituições.
- Ausência de Dinamismo: Os sistemas físicos tradicionais carecem de flexibilidade ao nível da personalização, da jornada, do visitante e da integração nativa com recursos multimédia e estatísticos avançados.
- Complexidade de Desenvolvimento Customizado: O desenvolvimento alternativo de APIs e painéis administrativos de raiz (*from scratch*) para cada museu implicaria um tempo de computação e custos de desenvolvimento incomportáveis face aos prazos de mercado.

Neste contexto, o projeto foi desenhado para contornar estas barreiras através de um modelo Bring Your Own Device(BYOD). O Museum Mate foi conceptualizado como uma plataforma modular e flexível que resolve o problema em duas frentes: garante aos administradores a capacidade de gerir e atualizar centralizadamente todos os conteúdos e idiomas remotamente, enquanto faculta aos visitantes um acesso ágil à informação contextual diretamente nos seus dispositivos móveis pessoais.

3.3.3. Objetivo da aplicação

O principal objetivo do projeto Museum Mate consiste no desenvolvimento de uma aplicação móvel para dispositivos Android, concebida para funcionar como um assistente digital de visita em museus e outros espaços culturais. A aplicação pretende complementar a experiência do visitante através da disponibilização de conteúdos multimédia associados aos diferentes pontos de interesse existentes no espaço, promovendo uma interação mais informativa, personalizada e acessível, sem comprometer a experiência de observação e apreciação do património.

Para além de apoiar o visitante durante o percurso, a solução foi desenvolvida de forma a permitir uma gestão centralizada e dinâmica dos conteúdos, possibilitando que todas as informações disponibilizadas na aplicação possam ser atualizadas sem necessidade de alterações ao código da aplicação móvel.

3.3.4 Requisitos do Sistema

Após a definição dos objetivos do projeto, tornou-se necessário identificar os requisitos que a solução deveria cumprir para responder às necessidades dos visitantes e da administração do museu. Estes requisitos constituíram a base para o desenvolvimento da aplicação, orientando a implementação das diferentes funcionalidades e assegurando que o sistema respondesse aos objetivos inicialmente definidos.

Os requisitos foram divididos em duas categorias: requisitos funcionais, que descrevem os serviços e funcionalidades disponibilizados pelo sistema, e requisitos não funcionais, que definem as características de qualidade, desempenho e funcionamento esperadas para a aplicação.

Requisitos Funcionais

Os principais requisitos funcionais definidos para o Museum Mate foram os seguintes:

- **Consulta de Mapas e Pisos**: Disponibilização de um mecanismo de visualização interativa das plantas do museu, organizadas e segmentadas hierarquicamente por diferentes pisos e áreas físicas.

- **Apresentação de Pontos de Interesse**: Exibição gráfica e textual de dados detalhados relativos às áreas delimitadas e aos pontos de interesse específicos mapeados em cada planta.

- **Disponibilização de Conteúdos Multimédia**: Integração de recursos informativos multiformato (incluindo cadeias de texto descritivo, imagens estáticas e faixas de áudio-guia) associados a cada obra ou item do acervo.

- **Reprodução Síncrona de Áudio-Guia**: Execução de faixas de áudio explicativas em simultâneo com a consulta visual dos ecrãs de detalhe dos itens por parte do visitante.

- **Leitura Ótica de Códigos QR**: Captura, processamento e descodificação nativa de códigos QR em tempo real através do recurso à câmara fotográfica integrada no dispositivo móvel.

- **Submissão Manual de Códigos**: Permissão para a introdução por teclado de identificadores alfanuméricos curtos e únicos, correspondentes a itens ou obras específicas em exposição.

- **Circuitos de Visitas Guiadas**: Disponibilização de percursos e itinerários temáticos compostos por sequências predefinidas e ordenadas de pontos de interesse, estruturadas previamente pela administração.

- **Persistência de Áudio em Segundo Plano**: Garantia de continuidade na reprodução do áudio-guia através de um serviço de primeiro plano (*Foreground Service*), mantendo o fluxo ativo mesmo quando a aplicação é minimizada ou o ecrã do telemóvel é bloqueado.

- **Armazenamento e Saturação de Cache Offline**: Mecanismo de persistência local da totalidade dos dados informativos e multimédia no arranque da aplicação, viabilizando a operação ininterrupta do sistema na ausência de conectividade à Internet.

- **Internacionalização Dinâmica**: Suporte multilingue em tempo real que adapta dinamicamente toda a interface gráfica e os conteúdos informativos consoante o idioma selecionado (Português, Inglês, Espanhol e Francês).

- **Administração Centralizada de Conteúdos**: Disponibilização de um sistema de gestão de conteúdos (*Headless CMS*) que permite aos administradores criar, editar ou remover museus, áreas, itens, visitas e idiomas de forma unificada.

- **Monitorização Analítica de Interações**: Registo automatizado e assíncrono de eventos de utilização e métricas de navegação na aplicação para posterior tratamento estatístico.

- **Geração Automatizada de Sinalética Digital**: Emissão automática de códigos QR únicos contendo o identificador do item no backend, gerados imediatamente após a criação de uma nova obra no painel de gestão.
    - Leitura por Tecnologia NFC: Deteção e descodificação de tags de comunicação por campo de proximidade (NFC) associadas às obras, como alternativa à leitura ótica de código QR.
    - Assistência Conversacional Inteligente: Disponibilização de um assistente virtual, apoiado em inteligência artificial, capaz de responder em linguagem natural a questões sobre o catálogo do museu, no idioma ativo do visitante, e de recusar questões alheias ao âmbito da instituição.

Requisitos não funcionais

Os principais requisitos não funcionais foram os seguintes:

- **Compatibilidade com Android:** Garantia de que a aplicação móvel compila de forma nativa e executa de forma estável no ecossistema Android, assegurando a portabilidade de todas as funcionalidades desenvolvidas em Qt/QML.
- **Interface Intuitiva:** Desenvolvimento de um *design* visual ergonómico, fluido e de fácil compreensão, minimizando a curva de aprendizagem para utilizadores com diferentes níveis de literacia tecnológica.
- **Funcionamento Offline:** Capacidade de a aplicação manter a sua operação em pleno mesmo na ausência de conectividade à Internet, recorrendo à persistência e consumo dos dados armazenados na cache local.
- **Desacoplamento Linguístico:** Separação total entre o código-fonte da aplicação e os textos da interface, isolando as traduções em dicionários de dados externos para evitar alterações ao código sempre que se adicionem novos conteúdos.
- **Suporte Multilingue:** Habilidade do sistema para comutar e apresentar dinamicamente toda a interface gráfica e os conteúdos informativos em múltiplos idiomas, em tempo real, consoante a preferência do visitante.
- **Elevada Disponibilidade:** Garantia de que o ecossistema (incluindo o servidor e os serviços de backend) permanece operacional e acessível de forma contínua, minimizando períodos de inatividade ou falhas de comunicação.

- **Facilidade de Manutenção:** Estruturação modular do código-fonte e dos componentes reutilizáveis, permitindo efetuar correções de bugs, atualizações ou melhorias de forma ágil e isolada.
- **Escalabilidade:** Capacidade de a arquitetura suportar o crescimento do volume de dados, do número de museus cadastrados e do tráfego de utilizadores em simultâneo, sem perda de estabilidade estrutural.
- **Baixo Tempo de Resposta:** Otimização dos algoritmos e das transições de ecrãs no *frontend* para garantir uma navegação fluida, imediata e com latências impercetíveis durante o consumo de conteúdos.

3.3.5 Arquitetura Geral do Sistema

Antes do início da implementação, tornou-se necessário definir a arquitetura geral do sistema, estabelecendo a forma como os diferentes componentes iriam comunicar entre si e quais as responsabilidades atribuídas a cada um deles. Esta fase permitiu estruturar o projeto de forma modular, facilitando a sua manutenção, evolução e adaptação a diferentes contextos de utilização.

Atendendo aos requisitos definidos anteriormente, concluiu-se que uma arquitetura centralizada na gestão de conteúdos seria a solução mais adequada. Deste modo, optou-se por separar a aplicação móvel da camada de gestão da informação, permitindo que os conteúdos fossem administrados independentemente do ciclo de desenvolvimento da aplicação.

A solução desenvolvida é composta por quatro componentes principais: a aplicação móvel, responsável pela interação com o visitante; o sistema de gestão de conteúdos (CMS), utilizado para administrar toda a informação disponibilizada; o painel de análise estatística, destinado à monitorização da utilização da aplicação; e a infraestrutura de contentorização, responsável pela distribuição e execução consistente do sistema em diferentes ambientes.

Esta separação de responsabilidades permitiu desenvolver uma solução modular, escalável e facilmente reutilizável, possibilitando a atualização dos conteúdos sem necessidade de publicar novas versões da aplicação móvel.

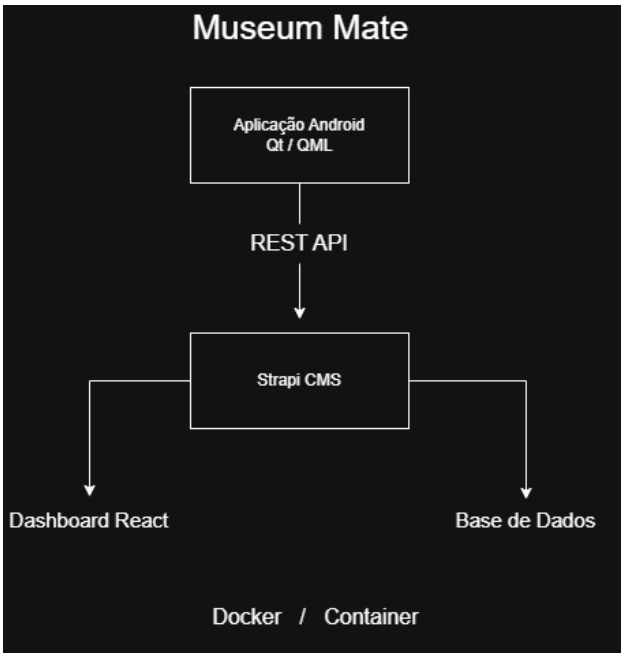


Figura 8 - Fluxo geral do Sistema

Um aspeto adicional da arquitetura, decorrente da necessidade de reutilizar o mesmo sistema para diferentes museus, foi a introdução de um único ponto de configuração que determina qual a entidade (museu) servida em cada momento, tanto pela aplicação móvel como pelo assistente virtual (secção 4.9). Esta decisão evita que a aplicação e o assistente possam, por lapso, apontar para museus diferentes, e reduz a troca de conteúdo a uma única alteração centralizada, sem necessidade de editar múltiplos ficheiros. Na prática, este mecanismo foi utilizado para alternar entre o museu fictício de demonstração, criado na fase inicial do projeto, e o conteúdo real de um museu, sem qualquer impacto na restante arquitetura.

3.3.6 Planeamento da Solução

Nesta fase, deu-se o arranque efetivo ao planeamento e à conceção do projeto principal, denominado *Museum Mate*. Sob a orientação direta do supervisor de estágio, o foco inicial centrou-se na definição clara do âmbito da aplicação, no levantamento detalhado dos requisitos funcionais e na estruturação lógica das fases de desenvolvimento subsequentes.

A abordagem metodológica adotada baseou-se no princípio de que uma análise arquitetural robusta deve anteceder qualquer esforço de programação. Deste modo, determinou-se a transposição das ideias e conceitos operacionais para um documento de especificações, mitigando riscos de desenvolvimento e organizando as prioridades do sistema, tais como a internacionalização dinâmica e o suporte a conteúdos multimédia.

Com o objetivo de validar a experiência do utilizador (*User Journey*) e estruturar a navegação da aplicação, utilizou-se a ferramenta de modelação *Draw.io*. Através desta plataforma, foi desenhado o fluxograma de navegação inicial, mapeando os pontos de decisão e a lógica de transição entre ecrãs, desde a seleção do idioma até à reprodução do áudio-guia interativo. Este mapeamento conceptual (ilustrado na Figura 9) revelou-se fundamental para definir a estrutura de dados a implementar posteriormente no *Headless CMS* Strapi.

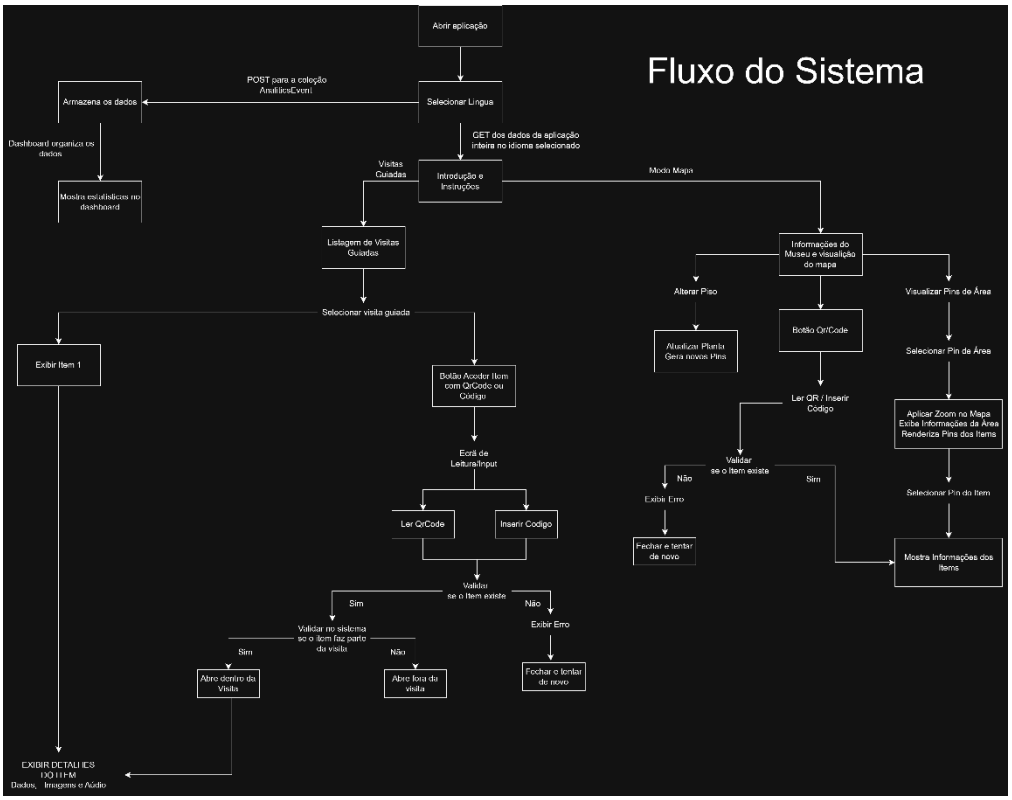


Figura 9 - Diagrama de fluxo de navegação do utilizador

Validada a componente funcional e o fluxo lógico de ecrãs, a fase seguinte consistiu na especificação e montagem da infraestrutura tecnológica de suporte, englobando a contentorização do ambiente de desenvolvimento e a modelação da base de dados.

3.3.7 Arquitetura Headless CMS com Strapi

Para fazer a gestão de dados do *Museum Mate*, decidi utilizar o Strapi, um *Headless* CMS de código aberto baseado em Node.js. A grande vantagem de usar o Strapi foi o facto de ele já trazer, por defeito, um painel de administração visual e muito intuitivo. Isto é ideal para o contexto do projeto, pois permite que pessoas sem conhecimentos técnicos ou de programação consigam ver, editar e gerir todos os dados da aplicação de forma simples, sem precisarem de mexer numa única linha de código.

Porquê escolher o Strapi e não uma base de dados tradicional?

A decisão de usar o Strapi em vez de criar uma base de dados tradicional do zero prendeu-se, sobretudo, com a produtividade e a facilidade de visualização. Se optasse por uma base de dados convencional, teria de programar não só a estrutura da base de dados, mas também toda a API para ligar ao telemóvel e uma interface de raiz para que os funcionários do museu pudessem inserir os conteúdos.

O Strapi resolveu tudo isto logo à partida, poupando imenso tempo de desenvolvimento. Além disso, ele não funciona como uma "caixa preta": por ser construído em Node.js no *backend* e React no *frontend*, é totalmente moldável. Esta flexibilidade deu-me a liberdade necessária para personalizar o sistema à nossa medida, separando por completo o armazenamento dos dados da interface gráfica que desenvolvi em QML.

Coleções criadas para armazenar os dados

Para organizar toda a informação da aplicação, estruturei o backend do Strapi em sete coleções (*Collection Types*). Cada uma tem uma função bem definida no ecossistema do Museum Mate:

Map	Tipo de dados
nameMap	Text
floorMap	Text
imageMap	Media
Maps N – 1 Entity	
Map 1 – N Areas	

Entity	Tipo de dados
nameEntity	Text
descriptionEntity	Text
omageEntity	Media
welcome_message	Text
audio	Media
Entity 1 – N Maps	

Area	Tipo de dados
nameArea	Text
subtitleArea	Text
descriptionArea	Text
imageArea	Media
audioArea	Media
posX	Number (decimal)
posY	Number (decimal
Area N – 1 Map	
Area 1 – N Items	

Item	Tipo de dados
nameItem	Text
descriptionItem	Text
code	Text
coverItem	Media
audioItem	Media
coverItem	Media
mediaItem	MultipleMedia
qrCodeImage	Media
poXItem	Number(decimal)
posYItem	Number(decimal)
Item N – 1 Area	

Tour	Tipo de dados
nameTour	Text
timeTour	Text
descriptionTour	Text
imageTour	Media
Tour 1 – N Items	

Dictionary	Tipo de Dados
key	Text
value	Text
Entity 1 – N dictionaries	

AnalyticsEvent	Tipo de dados
eventType	Enumeration
timestamp	Datetime
sessionID	Text
language	Enumeration
itemId	Number
tourId	Number
sucess	Boolean

Explicação de cada coleção

1. Entity (Entidade)

Criei esta coleção com o objetivo principal de armazenar os dados gerais da instituição (seja um museu, um jardim zoológico ou outro espaço cultural). A ideia aqui foi dar flexibilidade ao sistema: o *backend* consegue suportar diferentes entidades sem que seja necessário alterar a estrutura do código.

2. Map (Mapa)

Esta coleção serve para gerir os mapas interativos da aplicação e a paginação dos diferentes pisos ou edifícios que o espaço possa ter. É a base visual onde o utilizador se vai situar.

3. Area (Área)

Define zonas específicas dentro de cada mapa (como salas, galerias ou pavilhões). Funciona como um passo intermédio para agrupar os vários itens que estão expostos numa mesma divisão.

4. Item (Obra / Ponto de Interesse)

É o coração da base de dados, onde ficam guardadas as informações detalhadas de cada peça exposta. Para além do conteúdo descritivo e multimédia, inclui as coordenadas exatas para o posicionamento no ecrã, o código numérico de acesso rápido e o QR Code associado ao documentId.

5. Tour (Circuito / Visita Guiada)

O objetivo desta coleção é permitir que o administrador crie rotas temáticas. Quando um circuito novo é criado, o sistema filtra e interliga apenas os itens específicos que fazem parte desse caminho, para que o utilizador os possa seguir de forma sequencial.

6. Dictionary (Dicionário)

Foi a solução que encontrei para deixar a aplicação totalmente multi-idioma de forma dinâmica. Em vez de fixar (hardcoded) os textos e as palavras dos botões no código do telemóvel, esta coleção guarda as chaves e as respetivas traduções. Assim, o frontend consegue mudar o idioma da interface em tempo real com base nos dados do servidor.

7. AnalyticsEvent (Eventos Estatísticos)

Regista as interações dos utilizadores em segundo plano enquanto navegam pela app. Sempre que o visitante escolhe uma língua ou acede a uma obra, o sistema regista o evento de forma anónima. Estes dados brutos servem para alimentar diretamente o dashboard de estatísticas que integrei no Strapi.

Ferramenta de Posicionamento Visual de Itens e Áreas

Nos campos de coordenadas de cada Item e Area, a introdução manual de valores numéricos revelou-se pouco intuitiva para quem gere o conteúdo do museu, sobretudo para utilizadores sem conhecimentos técnicos. Para resolver esta limitação, foi desenvolvida uma extensão do painel de administração do Strapi que permite posicionar visualmente cada item ou área: o administrador visualiza a imagem do mapa correspondente e apenas tem de clicar no ponto exato onde a obra ou a sala se encontra, sendo as coordenadas calculadas e guardadas automaticamente pelo sistema. Esta ferramenta reforça o objetivo de tornar o Strapi acessível a pessoas sem formação técnica, reduzindo significativamente o risco de erro no posicionamento das obras no mapa.

Extensibilidade do CMS e Criação do Dashboard

Para complementar a gestão dos conteúdos disponibilizados pela aplicação, foi desenvolvido um sistema de analítica com o objetivo de recolher informação estatística sobre a utilização do Museum Mate. Esta funcionalidade permite à administração do museu compreender o comportamento dos visitantes, identificar os conteúdos com maior relevância e apoiar futuras decisões relacionadas com a organização das exposições ou com a evolução da própria aplicação.

Ao contrário de soluções baseadas apenas na contagem de acessos, o sistema desenvolvido foi concebido para registar eventos específicos que representam ações realizadas pelo utilizador durante a visita. Sempre que ocorre uma interação relevante na aplicação, é efetuado um pedido HTTP do tipo POST para

a API disponibilizada pelo Strapi, enviando um conjunto de informações que caracterizam o evento ocorrido.

Todos os eventos recebidos são armazenados na coleção AnalyticsEvent, criada especificamente para centralizar a informação estatística da aplicação. Cada registo corresponde a uma única ação executada pelo utilizador e contém um conjunto de atributos que permitem caracterizar essa interação, nomeadamente o tipo de evento, a data e hora de ocorrência, o idioma selecionado, o identificador do item ou da visita guiada, o resultado da operação efetuada e informação adicional armazenada em formato JSON.

Os principais eventos registados pelo sistema são:

- visit_start – início de uma nova visita e língua escolhida;
- item_view – visualização de um ponto de interesse;
- scan_success – leitura bem-sucedida de um código QR ou código numérico;

Decisão técnica dashboard integrado no Strapi

Os eventos registados na coleção AnalyticsEvent são posteriormente processados por um dashboard desenvolvido em React, integrado diretamente no painel de administração do Strapi. Esta integração foi possível devido à arquitetura open source e extensível do Strapi, que permite o desenvolvimento de plugins personalizados adaptados às necessidades do projeto.

Ao integrar o dashboard diretamente no Strapi, toda a componente de administração e monitorização passa a estar concentrada numa única plataforma. Para além disso, como o dashboard faz parte da própria instância do Strapi, a sua distribuição é igualmente assegurada através do ambiente Docker. Desta forma, sempre que o sistema é instalado ou restaurado, o backend, o dashboard e todas as configurações ficam imediatamente disponíveis, simplificando o processo de implementação.

O dashboard comunica com a API REST do Strapi para obter os eventos registados na coleção AnalyticsEvent, processando-os e apresentando-os de forma organizada ao administrador. Este dashboard permite consultar estatísticas por dia, semana, mês e ano, disponibilizando indicadores como o número de visitantes únicos, a evolução do número de visitantes relativamente ao período anterior, o idioma mais utilizado, a distribuição percentual dos idiomas e o item do museu com maior número de visualizações.

Foi algo fundamental para o projeto tendo em conta que os administradores do sistema valorizam analisar e validar como o sistema está se refletindo nos utilizadores, permitindo ter um feedback instantâneo do uso da aplicação.

Suporte Multilingue (i18n)

Um dos aspetos mais estratégicos da modelação foi a inclusão de suporte multilingue desde o início. Através do sistema de i18n do Strapi, cada conteúdo pode existir em múltiplos idiomas, identificado por diferentes *locales* (como “pt”).

Isto significa que:

- A mesma entidade pode ter versões em diferentes línguas
- O conteúdo é duplicado e adaptado por idioma
- A estrutura mantém-se, apenas variando os dados

Esta decisão garante que o sistema pode ser utilizado em contextos internacionais sem necessidade de alterações estruturais, algo que muitos sistemas não consideram inicialmente e acabam por ter dificuldade em integrar mais tarde. Exemplo de várias versões da coleção “Area” na figura 10.

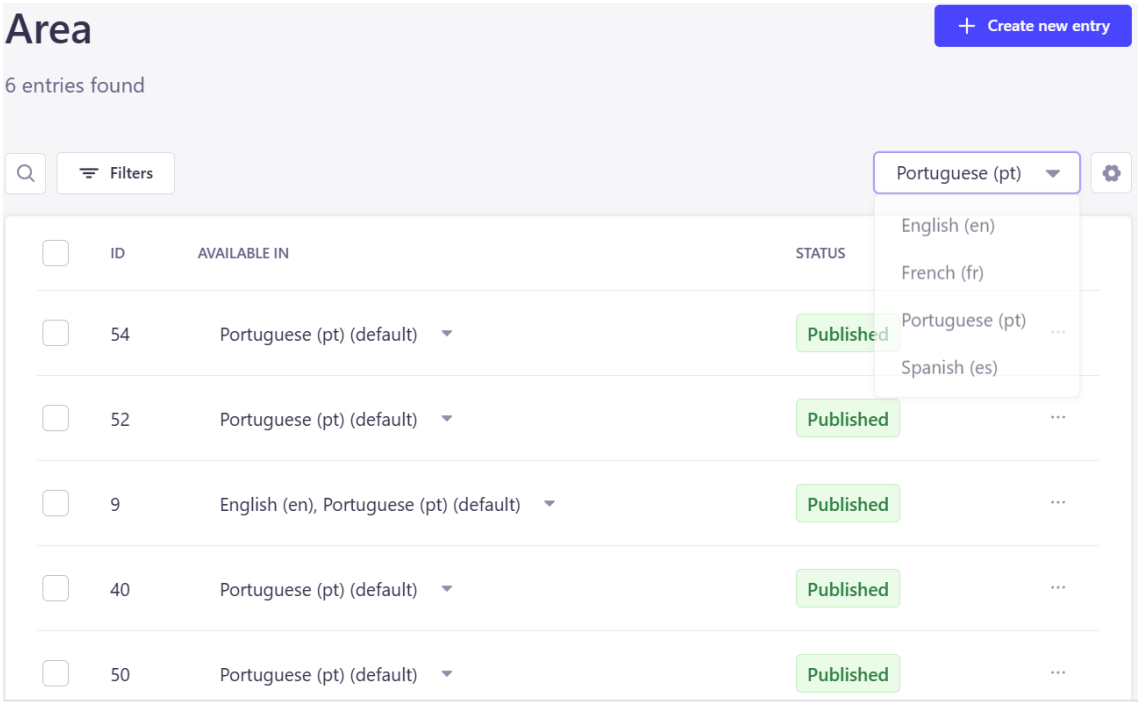


Figura 10 - Exemplo da coleção Área com o uso i18n

3.3.8. Contentorização e Infraestrutura com Docker

A componente de infraestrutura foi uma das decisões mais estratégicas no desenvolvimento do sistema. Mais do que uma escolha técnica, o foco foi o impacto direto na forma como o projeto pode ser distribuído, instalado e reutilizado. Desde cedo percebi que o grande desafio não era apenas fazer a aplicação funcionar na minha máquina, mas sim garantir a facilidade de replicação do sistema em qualquer outro ambiente.

Problema Inicial

Numa fase inicial, a execução do sistema dependia de várias configurações manuais no sistema operativo, o que introduzia muita complexidade e inconsistência. Entre os principais problemas que identifiquei estavam:

- Dificuldade em configurar o ambiente necessário para correr o Strapi de forma idêntica em computadores diferentes;
- Dependência da instalação manual do Node.js, gestão de versões e instalação de dependências;
- Erros inesperados ao tentar executar o projeto noutras máquinas devido a incompatibilidades de software;
- Risco de perda de dados ou inconsistência nos conteúdos inseridos entre diferentes ambientes de desenvolvimento.

Esta situação tornava o sistema pouco prático fora do meu computador de trabalho, o que ia contra o objetivo de criar uma solução reutilizável e facilmente distribuível para os museus.

A Decisão de Utilizar o Docker

O Docker surgiu precisamente como a resposta a este problema. A ideia foi permitir que o sistema possa ser executado em qualquer máquina sem necessidade de configurações manuais complexas. Desta forma, o Docker assume a responsabilidade de preparar todo o ambiente de forma isolada, libertando o administrador ou o utilizador dessa dor de cabeça.

Ao encapsular todo o ambiente dentro de um contentor (*container*), passou a ser possível:

- Garantir que o sistema funciona exatamente da mesma forma em qualquer computador;
- Eliminar a necessidade de instalar dependências externas no sistema anfitrião (*host*);
- Reduzir drasticamente o tempo de configuração inicial (*setup*);
- Garantir a persistência e a segurança dos dados.

Solução com Contentores e Evolução dos Scripts

A solução desenvolvida baseia-se num ambiente contentorizado que integra tudo o que é necessário para correr o *backend* com apenas um comando.

O ficheiro Dockerfile (Figura 12) define a imagem base com o Node.js, instala as dependências essenciais e prepara o sistema para a execução. Por sua vez, o docker-compose.yml (Figura 11) faz a gestão simplificada do serviço, controlando o mapeamento de portas para o acesso ao Strapi, a configuração das variáveis de ambiente e a definição de volumes para a persistência de dados.

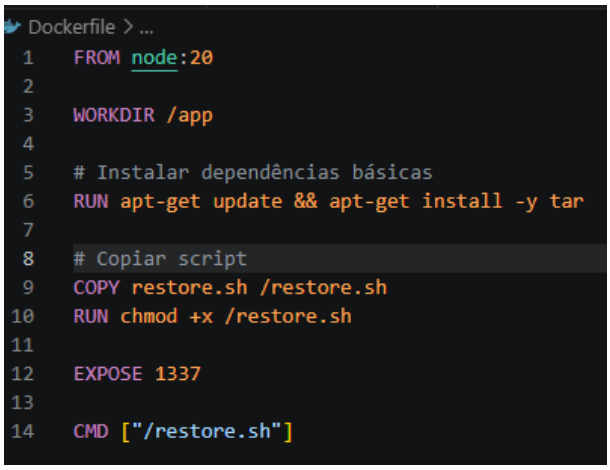


Figura 12 - Dockerfile

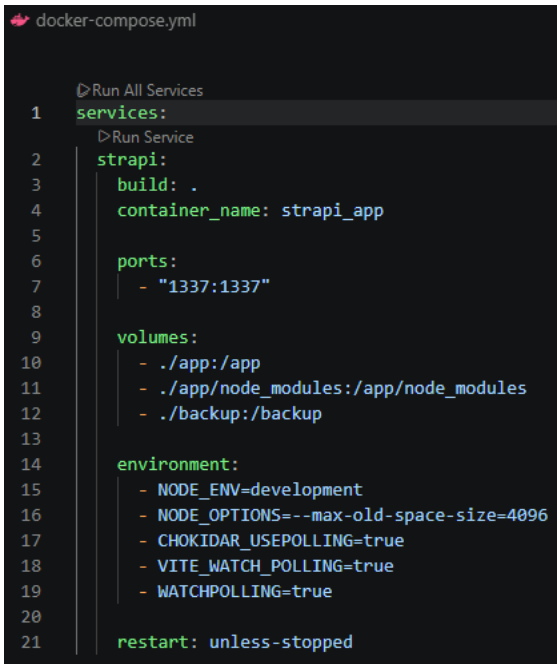


Figura 11 - docker-compose.yml

Automação com o restore.sh e Integração do Dashboard

Um dos elementos mais importantes desta infraestrutura é o script restore.sh (Figura 13), que foi criado para automatizar por completo o ciclo de vida da aplicação assim que o contentor arranca. Este script executa várias tarefas de forma sequencial e inteligente:

1. Verifica se o projeto Strapi já existe na pasta partilhada; caso não exista, cria-o automaticamente;
2. Verifica a existência de um ficheiro de cópia de segurança (backup.tar.gz); caso exista, restaura todos os dados para o sistema de forma automática;
3. Garante que todas as dependências do Node.js estão instaladas e atualizadas;
4. Inicia o servidor em modo de desenvolvimento.

Inicialmente, este script tratava apenas da base de dados do Strapi. No entanto, à medida que o projeto avançou e surgiu a necessidade de desenvolver o *dashboard* de estatísticas em React, fiz várias alterações e melhorias nesta componente. Atualizei os scripts de configuração para garantir que, ao executar o comando docker compose up, o estado atual do *dashboard* também é totalmente restaurado

Além disso, ajustei o mapeamento de volumes para otimizar o fluxo de trabalho. Com esta melhoria, enquanto estou a desenvolver as componentes em React dentro do contentor, basta fazer um *refresh* à página no navegador para que a interface se atualize sozinha com as novas alterações de código (*hot-reload*), sem que seja necessário reiniciar o contentor.



```
$ restore.sh

#!/bin/sh

echo "🚀 Starting Strapi container..."

# Criar projeto se não existir
if [ ! -f "/app/package.json" ]; then
  echo "📦 Creating Strapi project..."
  npx create-strapi-app@latest /app \
    --quickstart \
    --no-run \
    --skip-cloud
fi

cd /app

# 🔒 TRAVA DE SEGURANÇA: Restaurar o backup apenas se o ficheiro existir E se for a primeira vez.
# Isto impede que o Docker destrua o teu código novo a cada reinicialização!
if [ -f "/backup/backup.tar.gz" ] && [ ! -f "/app/.restored_marker" ]; then
  echo "♻️ Restoring backup data..."
  tar -xzf /backup/backup.tar.gz -C /app
  touch /app/.restored_marker
  echo "✅ Backup successfully restored!"
fi

# Instalar dependências se necessário
if [ ! -d "node_modules" ] || [ ! -f "node_modules/.bin/strapi" ]; then
  echo "📥 Installing dependencies..."
  npm install
fi

# ⚡ AUTOMAÇÃO: Se a pasta do teu plugin existir, liga o Watch em background (&)
if [ -d "src/plugins/analytics-dashboard" ]; then
  echo "📊 Custom plugin detected! Starting watch mode in background..."
  (cd src/plugins/analytics-dashboard && npm run watch) &
fi

echo "🔥 Starting Strapi with Admin Watch..."
# Substituído por npx com a flag --watch-admin para ficar atento ao build do teu plugin
npx strapi develop --watch-admin
```

Figura 13 - restore.sh

Persistência e Sistema de Backup

Para resolver o problema crítico da volatilidade dos contentores (garantir que os dados não se perdem quando o contentor é desligado ou recriado), defini volumes específicos que apontam para fora do Docker:

- /app: onde se encontra o código-fonte e as coleções do Strapi;
- /backup: onde são armazenados os ficheiros de segurança comprimidos.

Complementarmente, o sistema de backup gera um ficheiro backup.tar.gz. Como o restauro é feito de forma automática pelo script no arranque, o estado completo do sistema (dados, imagens e o próprio dashboard) pode ser transportado e replicado noutra máquina em poucos segundos.

Vantagens da Solução

A implementação do Docker trouxe melhorias gigantescas ao projeto, especialmente no que toca à usabilidade e manutenção:

- **Facilidade de instalação:** o sistema corre com um único comando (docker compose up);
- **Portabilidade:** funciona em qualquer sistema operativo que tenha o Docker instalado;
- **Consistência:** elimina o "na minha máquina funciona", garantindo um comportamento idêntico em qualquer lado;
- **Produtividade:** o fluxo com *live-reload* acelerou o desenvolvimento do painel em React.

Acima de tudo, a principal vantagem é a capacidade de distribuir o *Museum Mate* de forma simples, permitindo que qualquer utilizador ou equipa consiga colocar o sistema a funcionar sem precisar de conhecimentos técnicos avançados de *backend.*

Geração Automática de Códigos QR

Uma das funcionalidades desenvolvidas durante a implementação do backend consistiu na automatização da geração de códigos QR associados a cada item do museu. Esta funcionalidade surgiu da necessidade de simplificar o processo de gestão de conteúdos e reduzir a intervenção manual por parte do administrador do sistema.

Numa abordagem tradicional, após a criação de um novo item seria igualmente necessário gerar o respetivo código QR utilizando uma ferramenta externa, proceder ao seu armazenamento e, posteriormente, associá-lo manualmente ao registo correspondente. Para além de tornar o processo mais demorado, esta abordagem aumentaria a probabilidade de ocorrência de erros, como associações incorretas entre códigos QR e itens ou a inexistência do respetivo código.

Com o objetivo de eliminar estas limitações, foi desenvolvido um mecanismo automático integrado no ciclo de vida das entidades do Strapi. Sempre que um novo item é criado, é desencadeado um evento (*afterCreate*), responsável por executar automaticamente todo o processo de geração do código QR.

Durante este processo, o sistema utiliza o identificador único (*documentId*) do item para gerar uma imagem correspondente ao respetivo código QR. Após a sua criação, a imagem é temporariamente armazenada, enviada para a

biblioteca de ficheiros do Strapi através do serviço de *upload* e associada automaticamente ao registo do item recém-criado. Por fim, os ficheiros temporários utilizados durante a operação são removidos, garantindo a correta gestão dos recursos do sistema.

3.3.9 Metodologia de Desenvolvimento: Spec-Driven Development

À medida que o projeto Museum Mate crescia em número de funcionalidades e em complexidade — envolvendo dois stacks tecnológicos distintos e requisitos que foram sendo ajustados ao longo do estágio —, tornou-se cada vez mais difícil manter as decisões tomadas e os requisitos definidos apenas na memória ou em notas dispersas. Sendo um projeto desenvolvido individualmente, sem uma equipa a validar decisões em tempo real, a ausência de um processo estruturado representava um risco real de retrabalho e de perda de rastreabilidade sobre o porquê de cada escolha.

A Decisão de Adotar uma Metodologia Orientada por Especificações

Para mitigar este risco, adotei uma metodologia de desenvolvimento orientada por especificações (Spec-Driven Development), segundo a qual nenhuma funcionalidade nova é implementada sem que exista primeiro um documento de especificação que descreva o problema, os cenários de utilização e os critérios de aceitação. Esta abordagem alinha-se com o princípio já seguido no planeamento inicial do Museum Mate (secção 3.3.6), segundo o qual uma análise robusta deve sempre anteceder qualquer esforço de programação — a diferença é que passou a ser aplicada de forma sistemática a cada nova funcionalidade, e não apenas na conceção inicial do projeto.

Associada a esta metodologia, recorri ao Claude Code, um assistente de programação apoiado em inteligência artificial integrado diretamente no ambiente de desenvolvimento. A decisão de o utilizar prendeu-se com a necessidade de acelerar tarefas de análise de código, implementação e documentação, sem abdicar da validação humana em nenhuma etapa do processo.

Implementação e Funcionamento

O fluxo de trabalho adotado para cada funcionalidade seguiu sempre a mesma sequência: em primeiro lugar, era escrita uma especificação funcional, descrevendo o problema a resolver e organizando os cenários de utilização por ordem de prioridade; sempre que existiam pontos ambíguos, estes eram esclarecidos através de perguntas e respostas registadas na própria especificação, antes de se avançar. Só depois de a especificação estar validada era elaborado um plano técnico e uma lista de tarefas concretas, que orientavam a implementação passo a passo.

O Claude Code foi utilizado ao longo de todo este ciclo: na escrita e revisão das especificações, na implementação do código de acordo com as tarefas definidas, na análise de partes já existentes da aplicação antes de introduzir alterações, e na produção de documentação técnica de apoio. Em todos os casos, as sugestões geradas foram sempre revistas e validadas antes de serem aceites — o processo nunca dispensou a decisão final do autor.

Um exemplo concreto desta dinâmica ocorreu durante o desenvolvimento do assistente virtual do museu (secção 4.9): a funcionalidade de resposta a perguntas fora do catálogo foi inicialmente planeada com recurso a pesquisa externa na Internet. Testes realizados durante a implementação revelaram que esta opção exigiria ativar faturação na conta associada à chave de acesso ao serviço de inteligência artificial utilizado. Perante esta limitação, e tratando-se de um projeto de âmbito académico, optei conscientemente por não ativar essa faturação, adaptando à funcionalidade para que o assistente responda, nestes casos, com base no seu conhecimento geral, identificando sempre essa resposta como não verificada face ao catálogo oficial do museu.

Vantagens Obtidas

Esta metodologia trouxe maior rastreabilidade às decisões tomadas ao longo do estágio — cada funcionalidade ficou associada a um documento que explica não só o que foi implementado, mas também porquê — e tornou os requisitos explícitos antes da implementação, reduzindo o risco de retrabalho por más interpretações. O recurso ao Claude Code acelerou tarefas repetitivas de implementação e documentação, mas exigiu sempre validação crítica de cada sugestão gerada, já que a responsabilidade final por cada decisão de implementação continuou a ser exclusivamente minha. Em suma, a combinação entre especificações formais e assistência de IA contribuiu para um desenvolvimento mais disciplinado e mais bem documentado, sem nunca substituir o julgamento humano sobre o que efetivamente entra na aplicação final do museu.

4 Desenvolvimento do Frontend

Após a implementação da arquitetura do backend, iniciou-se o desenvolvimento da aplicação móvel Museum Mate. Esta componente é responsável por toda a interação entre o visitante e o sistema, permitindo o acesso aos conteúdos disponibilizados pelo Strapi através de uma interface desenvolvida em Qt/QML.

Ao longo desta secção serão apresentadas as principais páginas que constituem a aplicação, bem como o respetivo funcionamento e as decisões técnicas tomadas durante o seu desenvolvimento. Sempre que pertinente, serão igualmente descritos os mecanismos internos responsáveis pela comunicação com o backend, armazenamento local de dados, reprodução de conteúdos multimédia e recolha de estatísticas de utilização.

4.1.1 Arquitetura de Comunicação: Estratégia de Endpoint Único

No desenho da arquitetura de comunicação com o backend, foi tomada uma decisão estratégica fundamental: a centralização do consumo de dados num único endpoint unificado, em detrimento da utilização de múltiplos endpoints granulares para cada ecrã ou componente da aplicação.

Através do endpoint /api/entidades, e tirando partido do sistema de filtragem e parametrização profunda do Strapi, é injetada uma query string complexa que instrui o servidor a resolver e devolver toda a árvore relacional da aplicação (Entidades, Dicionários, Mapas, Áreas, Itens, QR Codes e Percursos/Tours) numa única transação HTTP.

Esta abordagem monolítica de transporte de dados oferece três vantagens críticas para o projeto:

- Minimização do *Network Chatter*: Reduz drasticamente o número de pedidos HTTP concorrentes, atenuando o impacto da latência da rede no dispositivo móvel.
- Consistência Atómica dos Dados: Garante que todo o conteúdo estrutural, multimédia e linguístico da aplicação é recebido em simultâneo, evitando estados de inconsistência visual durante a navegação.
- Sinergia Perfeita com o Modelo Offline: Facilita a hidratação imediata da base de dados local, permitindo que um único download síncrono popule toda a cache do dispositivo para o idioma selecionado.

A implementação prática desta lógica está utilizando este endpoint único:

```
http://IP-REDE:1337/api/entidades?locale=pt&populate[dictionaries]=true
&populate[imageEntity]=true&populate[audio]=true
&populate[tours][populate][imageTour]=true
&populate[tours][populate][items][populate][coverItem]=true
&populate[tours][populate][items][populate][audioItem]=true
&populate[maps][populate][imageMap]=true
&populate[maps][populate][areas][populate][imageArea]=true
&populate[maps][populate][areas][populate][audioArea]=true
&populate[maps][populate][areas][populate][items][populate][coverItem]=true
&populate[maps][populate][areas][populate][items][populate][audioItem]=true&
populate[maps][populate][areas][populate][items][populate][qrCodeImage]=true
```

Analiticamente, o parâmetro populate estende-se de forma multidimensional, forçando o Strapi a realizar os *joins* relacionais necessários no lado do servidor. O resultado mapeado em response.data é um objeto JSON que contém a totalidade do ecossistema do museu, pronto a ser serializado e injetado diretamente na persistência local.

4.1.2 Gestão de Dados, Cache e API (ApiHandler.js)

O ficheiro ApiHandler.js constitui o motor de dados central da aplicação Museum Mate. Em vez de sobrecarregar os componentes visuais do QML com lógica de comunicação em rede, optou-se por isolar todas as interações HTTP (pedidos *GET*) e o processamento de dados num módulo JavaScript dedicado. Este script funciona como uma camada de abstração entre a API REST do Strapi CMS e a interface do utilizador. O gráfico da figura 14 mostra como este motor de dados funciona.

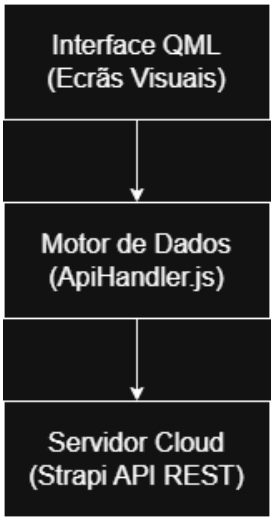


Figura 14 - Fluxo de comunicação entre a interface da aplicação e o servidor

4.1.3 Mecanismo de Persistência e Cache Local (*Offline-First*)

Uma das maiores restrições técnicas identificadas no ambiente de um museu é a instabilidade ou a ausência total de ligação à Internet em determinadas salas, frequentemente isoladas por paredes espessas. Para contornar este problema e garantir uma experiência contínua ao visitante, adotou-se uma estratégia de funcionamento focada na autonomia do dispositivo (*Offline-First*), tirando partido de um sistema de base de dados local instalado no telemóvel.

O ciclo de vida dos dados foi estruturado através de uma lógica de verificação inteligente:

1. **Verificação de Cache:** Sempre que a aplicação necessita de apresentar informações ao utilizador, uma função interna consulta primeiro a base de dados local do dispositivo para verificar se já existem dados guardados para o idioma selecionado.

2. **Carregamento Instantâneo:** Caso essa cópia local seja encontrada (cache hit), os dados são imediatamente convertidos e enviados para a interface. Isto garante que, a partir da segunda utilização, o arranque da aplicação é instantâneo e esta funciona de forma totalmente offline, sem qualquer dependência da rede.

3. **Sincronização em Segundo Plano:** Em paralelo, ou caso o dispositivo ainda não tenha dados guardados (como acontece na primeira utilização, onde a ligação à Internet é obrigatória), o sistema realiza um pedido assíncrono ao servidor.

4. **Atualização Transparente:** Assim que o servidor responde com sucesso, a base de dados local do telemóvel é atualizada e substituída pelos dados mais recentes. Este processo garante que o utilizador usufrui de uma navegação fluida, enquanto o sistema se mantém atualizado sempre que deteta rede.

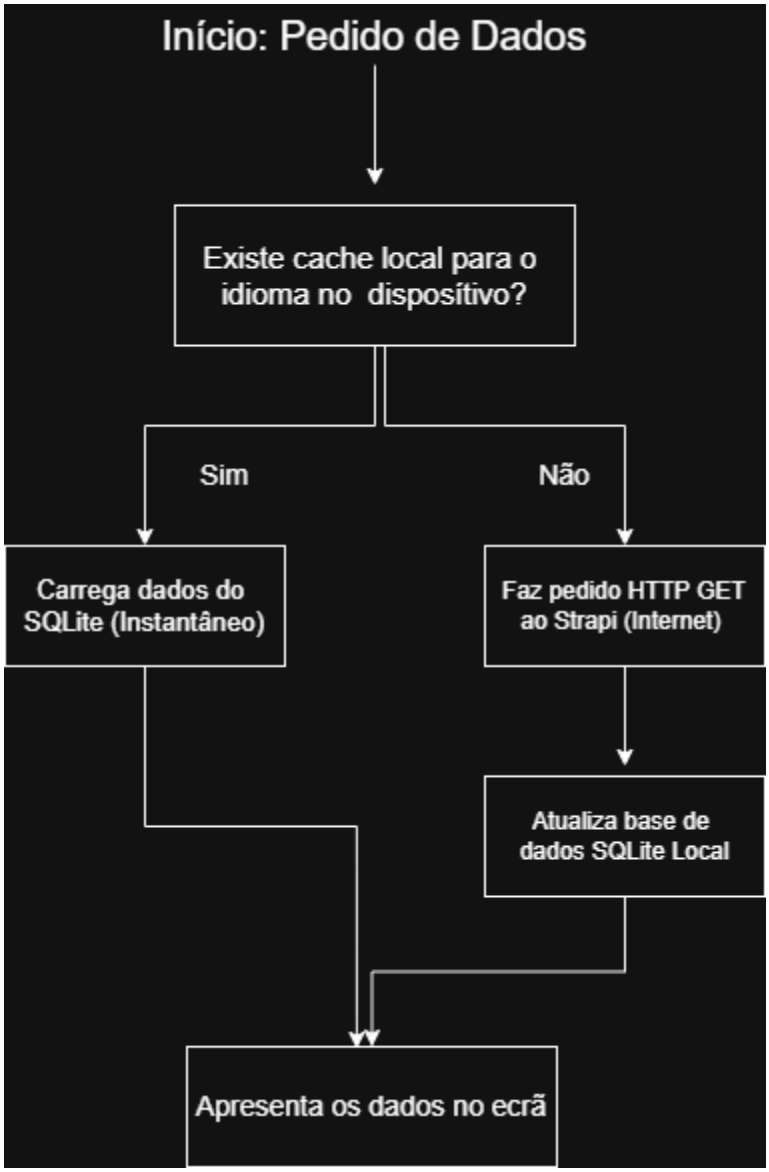


Figura 15 - Fluxo de Dados e Cache

4.1.4 Resolução Dinâmica de Recursos Multimédia

A gestão de ficheiros de média — tais como os áudios dos guias, as imagens de cobertura das obras e os mapas das salas — apresentou um desafio técnico acrescido. Ao contrário das descrições textuais, estes ficheiros físicos de grande dimensão não podem ser guardados diretamente como texto na base de dados local. Para resolver esta limitação, desenvolveu-se uma lógica de decisão automatizada para mapear a origem destes recursos ilustrada na figura 16:

Cenário de Primeiro Arranque (Online): Quando o utilizador instala e abre a aplicação pela primeira vez, o sistema deteta que os ficheiros físicos ainda não existem no telemóvel. A rota de leitura é direcionada para o endereço do servidor online, permitindo que a aplicação consuma as imagens e áudios diretamente da rede. Simultaneamente, um mecanismo auxiliar faz o descarregamento de todos estes ficheiros multimédia e guarda-os de forma organizada na memória interna do dispositivo.

Cenário de Visitas Subsequentes (Offline): Nas utilizações seguintes, a função encarregue de processar os caminhos dos ficheiros valida que os mesmos já se encontram armazenados localmente. O sistema altera dinamicamente a origem do recurso, isolando apenas o nome do ficheiro e apontando para a pasta segura no armazenamento do telemóvel.

Esta decisão de engenharia assegura que os ecrãs visuais e os leitores de áudio da aplicação permaneçam abstratos: eles não precisam de saber se o recurso provém da Internet ou da memória do telemóvel. O sistema resolve essa árvore de caminhos de forma invisível para o utilizador, salvaguardando a reprodução contínua dos áudio-guias mesmo em locais sem qualquer cobertura de rede.

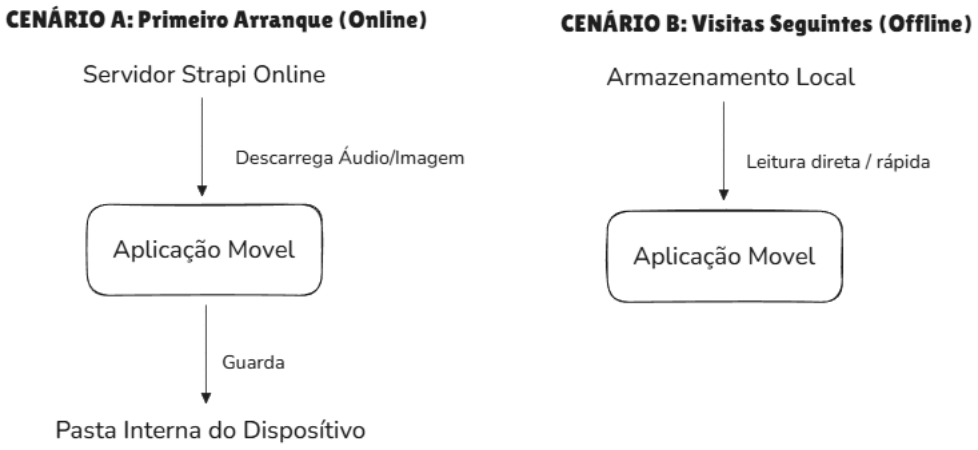


Figura 16 - Fluxo de dados Multimédia

4.2 Módulo de Telemetria e Monitorização (AnalyticsHandler.js)

A compreensão do comportamento do visitante dentro do museu — quais as obras mais visitadas, os percursos mais escolhidos, os idiomas preferidos e a taxa de sucesso na leitura de QR Codes — é fundamental para a otimização contínua da experiência museológica.

Para responder a esta necessidade sem comprometer o desempenho da interface, foi desenvolvido o módulo AnalyticsHandler.js. Este script é o único responsável por capturar, estruturar e encaminhar os dados comportamentais para a coleção AnalyticsEvent no Strapi CMS. À semelhança do motor de dados principal, este módulo opera sob o paradigma *offline-First*, garantindo que nenhuma métrica é perdida devido à instabilidade da rede nas salas do museu.

4.2.1 Fluxo de Processamento e Resiliência de Dados

O grande desafio técnico de um sistema de telemetria móvel em espaços fechados é evitar a perda de dados quando a ligação falha. O diagrama textual abaixo (figura 17) ilustra o algoritmo de decisão implementado no ciclo de vida de um evento:

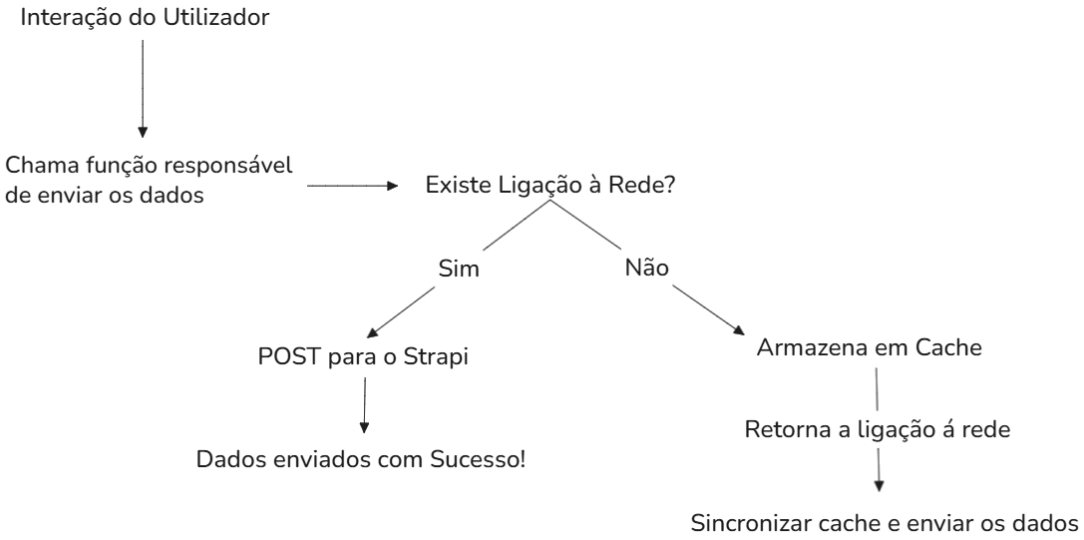


Figura 17 - Fluxo lógico do sistema de estatísticas

4.3 Janela Principal e Controlo de Estado (main.qml)

O ficheiro main.qml constitui o ponto de entrada da aplicação Museum Mate. Para além de apresentar o ecrã inicial de seleção de idioma, este componente é responsável por inicializar a aplicação, carregar todos os dados necessários ao seu funcionamento, iniciar a sessão do visitante e disponibilizar os serviços globais utilizados pelos restantes ecrãs.



Figura 18 - Página Inicial

4.3.1 Carregamento e Descarregamento Centralizado de Dados

Uma das principais decisões arquiteturais consistiu em centralizar todo o consumo de dados num único ponto da aplicação. Em vez de forçar cada ecrã a realizar pequenos pedidos HTTP individuais à medida que o utilizador navega, toda a informação estrutural e multimédia é processada logo no arranque.

Para materializar esta lógica, foi desenvolvida a função carregarDados(), cujo excerto técnico se apresenta abaixo na figura 19:

```
function carregarDados(lingua, passarPagina) {
    isLoading = true

    AH.getData(lingua, function(dados) {

        var mediaRaw = AH.extractMediaUrls(dados)
        var media = [...new Set(mediaRaw)]
        FileHelper.downloadFiles(media, AH.baseUrl, lingua)
        var localPath = FileHelper.getLocalMediaFolder(lingua)

        var novoModelo = {
            "entities": AH.getEntities(dados, localPath),
            "maps": AH.getMaps(dados, localPath),
            "areas": AH.getAreas(dados, localPath),
            "items": AH.getItems(dados, localPath),
            "dictionary": AH.getDictionary(dados, localPath),
            "tours": AH.getTours(dados, localPath),
            "tourItem": AH.getAllTourItems(dados, localPath)
        }
        appData = novoModelo
        isLoading = false
        if (passarPagina) {
            nextPage = 1
        }
    })
}
```

Figura 19 - função carregarDados

Vantagens obtidas:

- Disponibilidade Offline Imediata: Ao descarregar todos os recursos multimédia logo no arranque, os conteúdos ficam imediatamente guardados no telemóvel, permitindo o funcionamento estável da aplicação mesmo nas salas do museu sem cobertura de rede.
- Eficiência na Navegação: Como as restantes páginas nunca precisam de descarregar ficheiros ou fazer novos pedidos ao servidor, a alternância entre ecrãs é instantânea e o consumo de dados móveis é drasticamente reduzido.

4.3.2 Gestão da Sessão e Eventos Analíticos

A *main.qml* também controla as estatísticas da visita que depois aparecem no painel de administração do museu. No momento em que a aplicação acaba de abrir (*Component.onCompleted*), o sistema gera um identificador único de sessão (*sessionId*). Além disso, tenta enviar os dados guardados em cache através da função *Analytics.syncCache()*:

```
Component.onCompleted: {

root.sessionId = "SESS_" + Math.random().toString(36).substring(2, 11).toUpperCase() +
Date.now().toString(36).toUpperCase()

console.log("Nova sessão iniciada:", root.sessionId)

carregarDados(linguaEscolhida, false)

Analytics.syncCache() }
```

Este código cria um código de texto único (por exemplo, SESS_A1B2C3D4) que junta letras aleatórias com a hora exata do telemóvel. Ao contrário do ID do aparelho, este código identifica apenas aquela visita específica, permitindo saber o percurso que o utilizador fez no museu naquele dia. A chamada ao *syncCache*() serve para garantir que, se o utilizador acumulou dados offline numa visita anterior (por falta de rede), esses dados sejam enviados para o servidor logo no arranque da nova visita.

4.3.3 Navegação com o Componente Loader

Para alternar entre ecrãs sem deixar o telemóvel lento, a aplicação utiliza um componente chamado *Loader*. Esta lógica presente na figura 20 vigia a variável *nextPage* e carrega a página correspondente apenas quando ela é necessária.

```
Loader {
    id: gameLoader
    anchors.fill: parent
    active: nextPage !== 0
    opacity: active ? 1 : 0

    // Injetamos o objeto appData completo no WelcomePage
    sourceComponent: {
        if (nextPage === 1) return welcomePage
        return null
    }

    Behavior on opacity { NumberAnimation { duration: 500 } }
}

Component {
    id: welcomePage
    WelcomePage {
        appDataWelcome: root.appData
        linguaEscolhidaWelcome: root.linguaEscolhida
        onBack: nextPage = 0
    }
}
```

Figura 20 - Loader main

Esta forma de navegar traz duas grandes vantagens técnicas:

- **Poupança de Memória RAM:** As páginas secundárias da aplicação (como a página de boas-vindas ou a página do mapa) não ficam abertas ao mesmo tempo em segundo plano. Quando o utilizador muda de ecrã, o *Loader* destrói a página anterior e liberta a memória do telemóvel.
- **Injeção Direta de Dados:** Como se pode ver no bloco *welcomePage*, o objeto *appData* (que tem as informações guardadas) é passado diretamente para a nova página através da propriedade *appDataWelcome*. Graças a isto, a nova página já nasce com todos os dados de que precisa e não tem de fazer pedidos adicionais.

4.4 Ecrã de Boas-Vindas (WelcomePage.qml)

A *WelcomePage* serve como o primeiro ecrã de conteúdos que o visitante vê após selecionar o seu idioma na raiz da aplicação. O seu principal objetivo é funcionar como um ponto de triagem e encaminhamento. A página (figura 21) apresenta uma breve introdução sobre o museu e disponibiliza duas opções claras de navegação: iniciar uma Visita Livre (que abre o mapa interativo) ou aceder às Visitas Guiadas (circuitos de áudio).



Figura 21 - Página de boas-vindas

4.5 Ecrã Principal e Mapa Interativo (HomePage.qml)

A *HomePage* é a página de acolhimento e o ecrã principal de toda a aplicação. Funciona como um ponto de partida crucial, pois é o primeiro contacto visual que o visitante tem com o mapa digital do museu assim que decide iniciar a sua exploração livre.

4.5.1 Descrição Geral da Interface

O grande objetivo deste ecrã é responder de forma imediata a duas perguntas fundamentais de qualquer visitante: *"Onde é que eu estou?"* e *"O que é que há para ver aqui perto?"*. Para que isto aconteça de forma natural, a interface foi desenhada com um visual limpo e minimalista, focado na usabilidade, evitando sobrecarregar o utilizador com demasiado texto logo no primeiro instante.

Como se pode observar na Figura 22 a arquitetura visual da página divide-se em três zonas principais perfeitamente integradas:

- **O Mapa Dinâmico (Zona Central):** Onde é exibido o desenho técnico do piso selecionado. Em cima deste mapa, são desenhados marcadores visuais (*pins*) interativos que representam as diferentes salas ou áreas de exposição.
- **O Seletor de Pisos (Zona Superior/Lateral):** Um conjunto de botões simples que permite ao utilizador alternar instantaneamente entre os diferentes níveis físicos do museu (por exemplo, Piso 0 e Piso 1).
- **O Botão de Acesso Rápido ao Scanner (Zona Flutuante):** Um atalho sempre acessível que permite abrir a câmara do telemóvel num clique para ler os códigos QR espalhados pelo edifício, sem que o utilizador tenha de navegar por menus complexos.

Esta disposição garante que, mesmo um visitante sem grande experiência tecnológica, consiga perceber imediatamente que o mapa reage aos seus toques e que a aplicação está ali para servir de bússola digital durante toda a sua estadia.



Figura 22 - HomePage

4.5.2 Organização por Pisos e Filtragem de Dados

Para que o telemóvel não fique lento, a aplicação não carrega todas os pontos de interesse e obras, pecas, etc… ao mesmo tempo no ecrã. Se o fizesse, os ícones (*pins*) iriam sobrepor-se, criando uma enorme confusão visual.

A organização é feita através de uma separação automática: quando o utilizador muda de piso na aplicação (alterando a variável *currentFloor*), o sistema esconde os ícones do piso anterior e desenha apenas as áreas que pertencem ao piso atual.

O maior desafio nesta lógica foi a forma como os dados vêm do servidor. O servidor organiza as obras com base na sala a que pertencem, usando uma contagem local. Por isso, foi criada a função *updateFilteredItems*(). Esta função faz uma pesquisa rápida na memória do telemóvel para descobrir exatamente quais as obras que pertencem à sala onde o utilizador clicou, como se pode ver no código abaixo da figura 23:

```
function updateFilteredItems() {
    var allItems = appDataHomePage.items;
    var filtered = [];

    var targetFloor = Number(currentFloor);
    var globalAreaIndex = Number(selectedPinIndex);

    // 1. Calcular o índice relativo da área dentro do mapa atual
    var targetAreaRelative = 0;
    for (var a = 0; a < globalAreaIndex; a++) {
        if (Number(appDataHomePage.areas[a].mapIndex) === targetFloor) {
            targetAreaRelative++;
        }
    }

    // 2. Filtrar os itens comparando com o índice relativo correto
    for (var i = 0; i < allItems.length; i++) {
        var itemMapIndex = Number(allItems[i].mapIndex);
        var itemAreaIndex = Number(allItems[i].areaIndex); // Índice relativo do item

        if (itemMapIndex === targetFloor && itemAreaIndex === targetAreaRelative) {
            filtered.push(allItems[i]);
        }
    }
    filteredItems = filtered;
}
```

Figura 23 - Função updateFilteredItems

4.5.3 Ajuste Automático do Mapa no Ecrã

Quando o visitante clica no ícone de uma sala, o mapa faz uma aproximação automática (um Zoom de 2 vezes o tamanho normal). No entanto, como as caixas com as informações e textos sobre as obras ocupam muito espaço no ecrã, havia o risco de o mapa fazer zoom e a sala ficar escondida atrás do texto.

Para resolver este problema de design, foi criado um temporizador inteligente chamado *timerPositionMap*. Assim que o utilizador clica numa sala, este componente mede o tamanho do ecrã do telemóvel e decide o que fazer. Este fluxo segue-se desenhado na figura 24.

- Em Telemóveis (Ecrãs pequenos): O sistema verifica se a sala clicada está na parte de cima ou na parte de baixo do mapa. Se estiver em cima, o mapa desloca-se ligeiramente e a caixa de texto abre em baixo; se a sala estiver em baixo, o texto abre no topo do ecrã. Isto garante que a sala nunca fica tapada.
- Em Tablets ou Computadores (Ecrãs maiores): O mapa centra-se na sala clicada, mas aplica uma regra de segurança chamada *clamp* (limitação). Esta regra funciona como uma "barreira invisível" que impede o mapa de deslizar demasiado para os lados, evitando que o utilizador veja o fundo branco do ecrã.

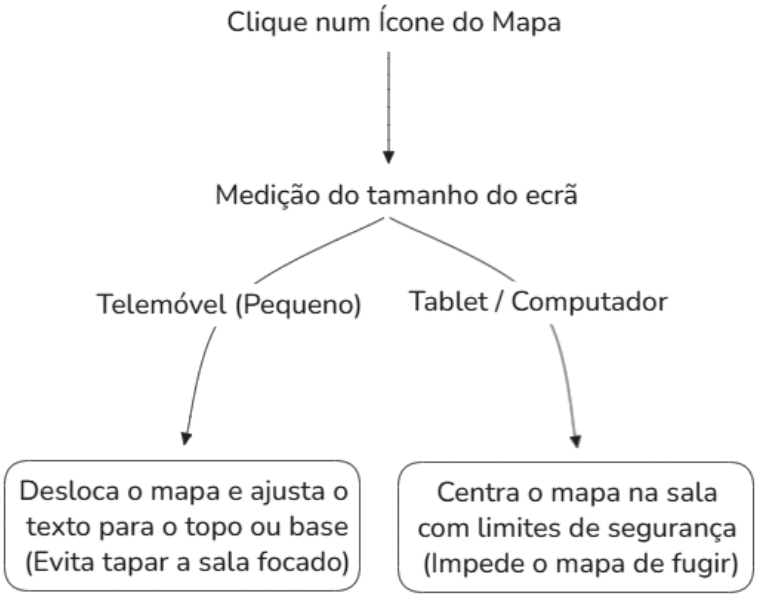


Figura 24 - Fluxo do timerPositionMap

O funcionamento desta função é evidenciado nas Figuras 25 e 26. A caixa de informação posiciona-se acima ou abaixo dos *pins*, dependendo da sua localização no ecrã, de modo a garantir que os elementos âncora nunca fiquem escondidos enquanto os detalhes do item são exibidos.



Figura 26 - Pins na zona Superior

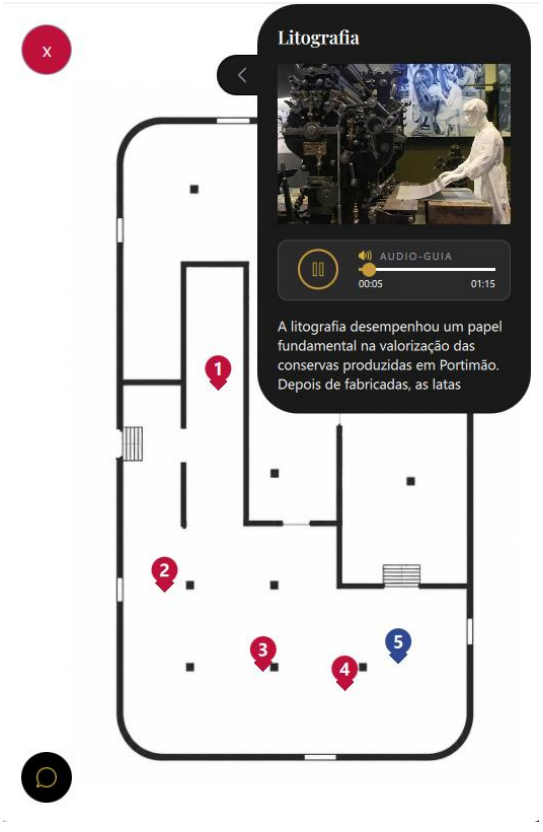


Figura 25 - Pins na zona inferior

4.5.4 Leitura de Códigos QR com a Biblioteca QZXing

Para além da navegação através do toque no ecrã, o visitante pode simplesmente apontar a câmara do telemóvel para os códigos QR salvaguardados nas salas ou nas etiquetas físicas das próprias obras do museu. Para que isto fosse possível, foi integrada na HomePage uma ferramenta de leitura chamada QZXing (uma biblioteca open-source bastante otimizada para detetar e descodificar imagens de códigos de barras e códigos QR no ecossistema Qt).

O funcionamento deste sistema é simples: a aplicação ativa o fluxo de vídeo da câmara e a biblioteca QZXing analisa os fotogramas em tempo real. Assim que deteta um código QR válido, a biblioteca converte essa matriz de imagem numa cadeia de texto (*string*) que contém o código identificador da obra — correspondendo ao documentId ou ao número interno da peça.

Com este código decifrado, a aplicação realiza uma pesquisa instantânea na lista de dados que guardou localmente no arranque. Se encontrar a peça correspondente, desliga de imediato a câmara para mitigar o consumo de bateria do telemóvel e ativa o protocolo de navegação automática. Todo este processo de transição, desde a leitura física até à resposta da interface, pode ser observado no esquema de funcionamento da Figura 27.

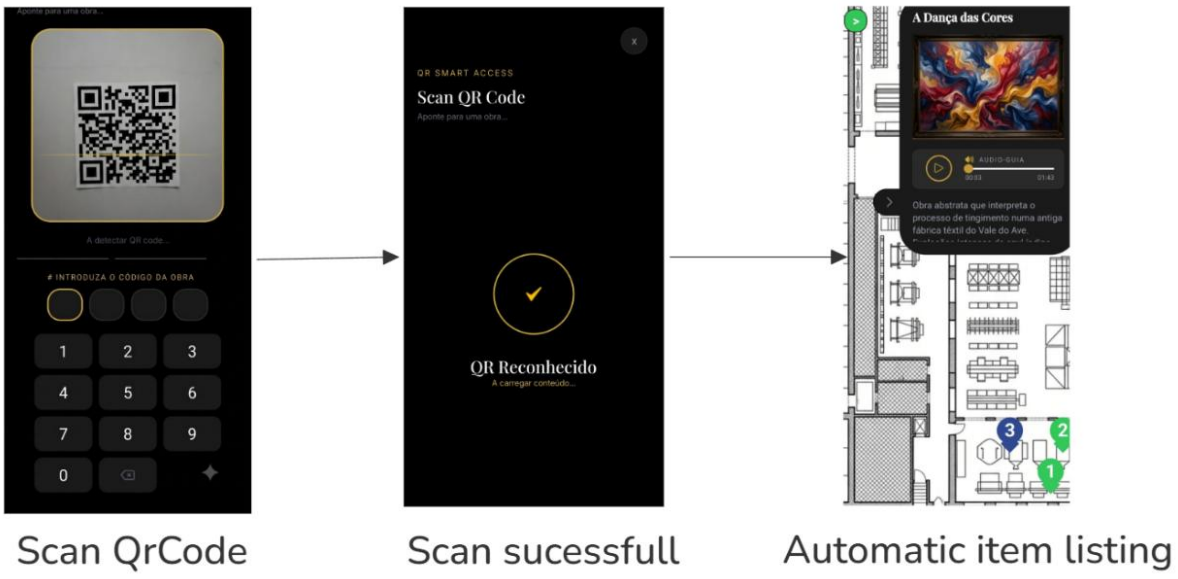


Figura 27 - Fluxo QrCode

Este mecanismo de leitura está também disponível a partir de outras páginas da aplicação através de um ecrã de scanner dedicado, que reúne ainda a introdução manual de código e a leitura por NFC como alternativas (secção 4.6).

4.5.5 Razões e Benefícios da Automação de Percurso

A criação de uma função centralizada para gerir a navegação automática não foi apenas uma decisão de programação, mas sim uma escolha estratégica para melhorar a experiência do visitante dentro do museu.

Num guia digital convencional, quando um visitante encontra uma obra de arte física e quer saber mais sobre ela, é obrigado a passar por um processo moroso: olhar para a aplicação, descobrir em que piso está, procurar a sala correta no mapa, adivinhar qual é o ícone da obra e, finalmente, clicar nele. Este excesso de passos gera frustração, cria distrações e afasta a atenção do utilizador daquilo que realmente importa: a arte.

Abaixo segue-se o fluxo real do funcionamento da leitura do qrCode na figura 28.

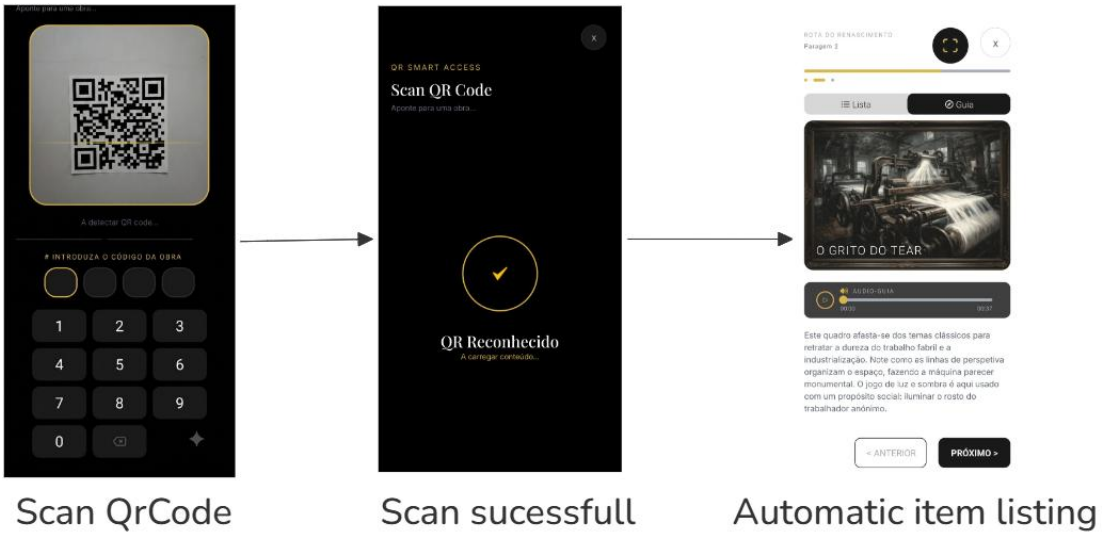


Figura 28 - Leitura e processamento automático do QR Code

Ao implementarmos este sistema automatizado, conseguimos eliminar por completo esta barreira. Os principais benefícios desta abordagem prática incluem:

- Eliminação do Atrito (Experiência Fluida): O utilizador só precisa de fazer uma única ação física — apontar a câmara para o código QR ou digitar quatro números. Num piscar de olhos, a aplicação encarrega-se de fazer todo o "trabalho pesado" de navegação por software, mudando o piso e fazendo o zoom correto no mapa de forma instantânea.
- Ponte Direta entre o Real e o Digital: Conforme demonstrado no fluxo da Figura 28 (no capítulo anterior), a aplicação liga o mundo físico ao digital sem interrupções. O visitante olha para a peça real e a aplicação reage de imediato, apresentando o texto explicativo e iniciando o áudio-guia correto sem necessidade de buscas manuais.

Em suma, esta automação transforma o telemóvel num assistente invisível e inteligente que responde de imediato ao que o visitante está a ver no mundo real.

4.6 Identificação de Obras: Scanner, Código Manual e NFC

Para além da navegação através do mapa interativo, a identificação de uma obra do museu está disponível a partir de um ecrã de scanner dedicado, acessível através de um botão flutuante presente em várias páginas da aplicação — nomeadamente na página de HomePage e durante uma visita guiada —, sem que o visitante tenha de regressar ao mapa para o efeito.

Este ecrã reúne três formas complementares de identificar uma obra, adequadas a diferentes condições da visita: a leitura ótica de um código QR através da câmara do dispositivo (seguindo o mesmo princípio já descrito na secção 4.5.4), a introdução manual de um código numérico de quatro dígitos, ou a deteção de uma tag de comunicação por campo de proximidade (NFC), bastando encostar o telemóvel junto à obra. Antes de ativar a câmara, a aplicação verifica se já existe autorização de acesso concedida pelo sistema operativo Android; caso contrário, a autorização é solicitada nesse momento, e é apresentado um estado alternativo caso o visitante a recuse, evitando um ecrã em branco sem explicação.

Independentemente do método utilizado, o comportamento final é o mesmo: o código identificado é procurado na base de dados local carregada no arranque da aplicação e, ao encontrar a obra correspondente, o sistema navega automaticamente até à sua informação, poupando ao visitante a necessidade de a localizar manualmente no mapa.

Ao centralizar as três formas de identificação num único ecrã, acessível a partir de várias páginas da aplicação, o visitante deixa de estar limitado a uma única

tecnologia ou a um único ponto de entrada: em espaços com pouca luminosidade ou obras próximas entre si, onde a leitura ótica é mais difícil, a alternativa por NFC ou por código manual garante que a experiência de identificação nunca fica bloqueada.

4.7 Lista de Visitas Guiadas (Tours.qml)

O ecrã Tours funciona como um catálogo interativo de visitas guiadas. O seu principal propósito é organizar e apresentar ao visitante os percursos temáticos disponíveis no museu (por exemplo, "Visita Histórica", "Grandes Obras de Arte" ou "Percurso dos Fundadores").

4.7.1 Descrição Geral e Propósito do Ecrã

Esta página foi desenhada para resolver um problema central na experiência do utilizador: a indecisão e a falta de tempo e a necessidade de visitas personalizadas para entidades como escolas e ATL. Ao condensar as visitas num formato de lista limpo e apelativo, o visitante consegue perceber de imediato, e num único relance, quanto tempo vai demorar cada percurso e quantas paragens inclui. Isto permite-lhe escolher a visita que melhor se adequa à sua disponibilidade, sem a necessidade de ler textos longos e confusos.

Como se ilustra na Figura 29, a interface é composta por um cabeçalho intuitivo e dinâmico



Figura 29 - Listagem de Visitas Guiadas

4.8 Ecrã da Visita Guiada Ativa (TourPage.qml)

O ecrã TourPage.qml representa o núcleo da experiência interativa da aplicação. Ao contrário de uma simples listagem estática, esta página foi desenhada e projetada especificamente para complementar a visita física do utilizador dentro do museu, fundindo de forma orgânica a componente visual com um sistema avançado de áudio-guia.

4.8.1 Descrição Geral e Objetivos de Conceção

O grande foco no desenvolvimento deste ecrã centrou-se na exploração sequencial e inteligente do espaço. Através da integração com um *Foreground Service* nativo do Android (cujo funcionamento estrutural e técnico é detalhado adiante, no Capítulo 4.7.2), a aplicação permite que o visitante oiça a descrição detalhada de uma determinada obra de arte e, de forma totalmente contínua, passe para o item seguinte da sala assim que a reprodução termina.

A arquitetura desta página ilustrada na figura 30 foi organizada para guiar o utilizador passo a passo, fornecendo inclusive informações contextuais e instruções adicionais de transição entre as peças expostas, garantindo que o visitante se consegue localizar facilmente no espaço físico do museu sem quebrar a sua imersão cultural.

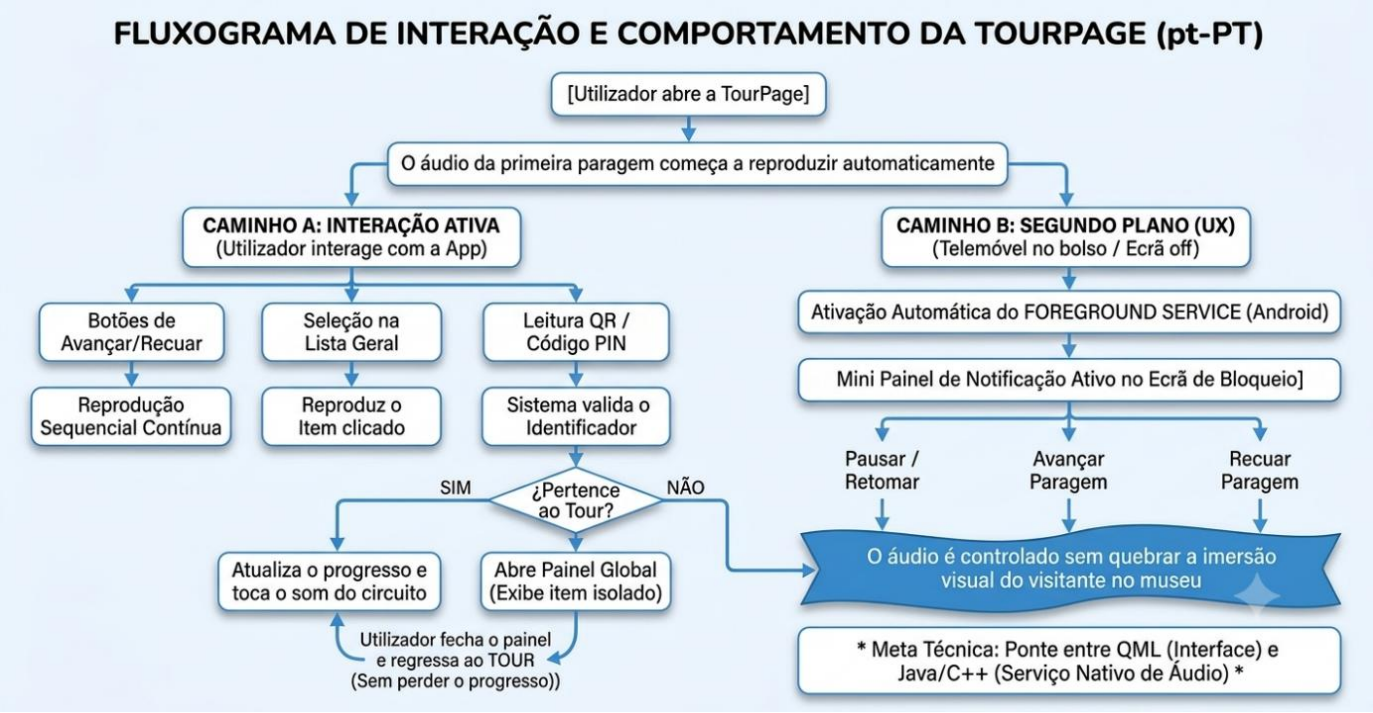


Figura 30 - Fluxo da Página TourPage.qml

4.8.2 Controlo Nativo e Arquitetura do Foreground Service

Um dos maiores desafios técnicos no desenvolvimento de guias de museus para dispositivos móveis é a gestão de bateria e a interrupção do sistema operativo. Se a aplicação utilizasse o reprodutor de áudio padrão da interface (QtMultimedia no QML), a música pararia instantaneamente assim que o visitante bloqueasse o ecrã do telemóvel ou guardasse o aparelho no bolso.

Para resolver este problema crítico e permitir que o visitante explore o museu com as mãos livres, foi necessário abandonar o controlo de áudio da interface gráfica e transferir essa responsabilidade diretamente para o "motor" do Android. Isto foi alcançado através da criação de uma arquitetura em três camadas (Tri-Layer), que liga o QML (Interface) ao C++ (Ponte de Comunicação) e ao Java (Serviço Nativo).

A. O Fluxo Arquitetural (Tri-Layer)

Para garantir que o áudio e a interface gráfica não bloqueiam um ao outro, a aplicação utiliza um padrão de comunicação bidirecional ilustrado no esquema abaixo (Figura 31):

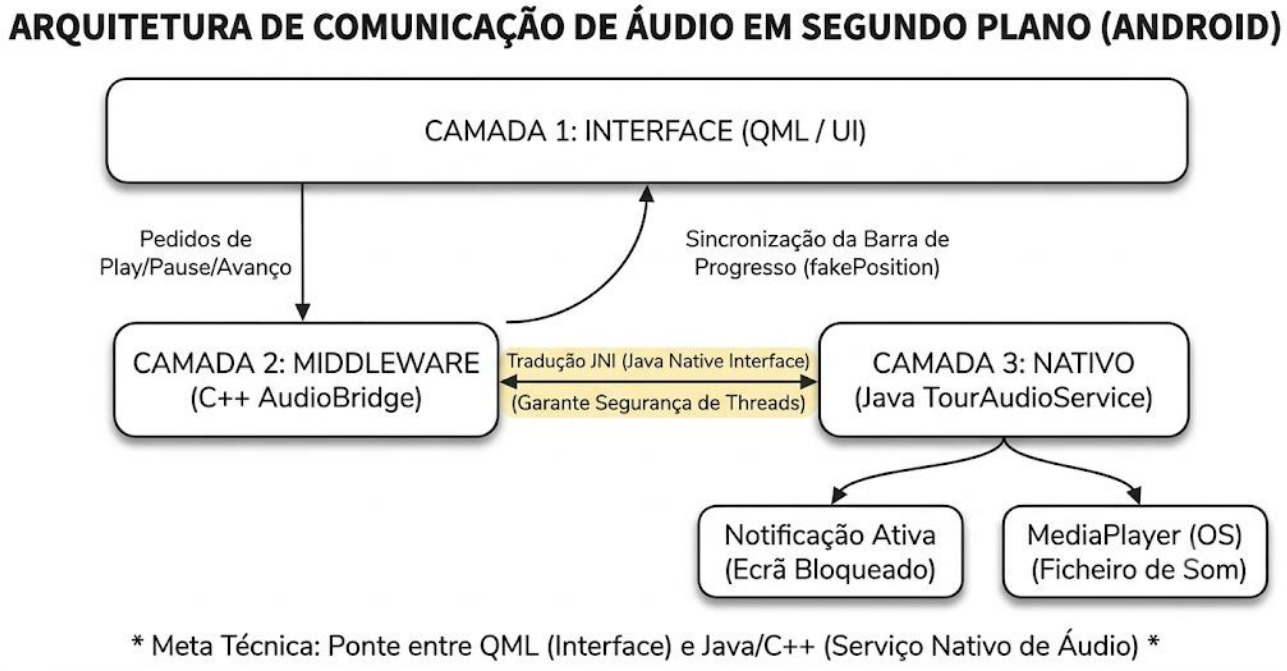


Figura 31 - Arquitetura Foreground Service

B. A Camada Nativa (Java): *TourAudioService*

A base desta arquitetura reside no ficheiro TourAudioService.java. Em vez de ser apenas um reprodutor de som em memória volátil, esta classe herda a propriedade fundamental Service do ecossistema Android. Isto permite que o sistema operativo a promova a um Foreground Service (Serviço de Primeiro Plano), blindando o processo contra encerramentos abruptos quando o telemóvel entra em modo de poupança de energia.

No ciclo de vida do serviço, a sua inicialização estabelece as duas pontes cruciais de interatividade exigidas: a receção de comandos de hardware (auscultadores e botões Bluetooth) e o controlo nativo em segundo plano através de uma *MediaSessionCompat*. A função responsável por executar esta logica segue-se representada abaixo na figura 32

```java
// Excerto simplificado de TourAudioService.java
public class TourAudioService extends Service {
    private static MediaPlayer player;
    private MediaSessionCompat mediaSession;

    @Override
    public void onCreate() {
        super.onCreate();
        sInstance = this;

        // Inicialização do barramento de comandos de hardware externos (Bluetooth/Auscultadores)
        mediaSession = new MediaSessionCompat(this, "MuseumAudioSession");
        mediaSession.setActive(true);
        mediaSession.setCallback(new MediaSessionCompat.Callback() {
            @Override
            public void onPlay() { handleAction(ACTION_PLAY); }
            @Override
            public void onPause() { handleAction(ACTION_PAUSE); }
            @Override
            public void onSkipToNext() { handleAction(ACTION_NEXT); }
            @Override
            public void onSkipToPrevious() { handleAction(ACTION_PREVIOUS); }
            @Override
            public void onSeekTo(long pos) { seekToPosition((int) pos); }
        });
    }
}
```

Figura 32 - Lógica da TourAudioService.java

C. A Ponte de Comunicação (C++): AudioBridge

A linguagem interpretada QML (responsável pela interface gráfica) e a linguagem Java (responsável pelo motor do Android) operam em ecossistemas completamente isolados. Para que comuniquem sem fricção, a aplicação utiliza a classe C++ *AudioBridge* como um intermediário de conversão universal através de JNI (Java Native Interface).

O *AudioBridge* resolve o problema mais crítico no desenvolvimento para dispositivos móveis: o conflito de *Threads* (linhas de execução). O serviço Java monitoriza o estado do áudio numa linha de fundo assíncrona, enviando atualizações a cada 250ms. Se o Java tentasse injetar este valor diretamente na interface gráfica, ocorreria um colapso crítico (*crash* do motor de renderização).

Para blindar a aplicação, o C++ expõe uma função nativa ilustrada na figura 33 pura detetável pelo kernel do Android (extern "C"), que recebe a informação e a "re-despacha" em segurança para a linha principal do Qt através de um mecanismo de concorrência assíncrona (Qt::QueuedConnection):

```
extern "C" JNIEXPORT void JNICALL
Java_org_qtproject_example_TourAudioService_nativePositionChanged(
    JNIEnv *env, jobject thiz, jint position, jint duration)
{
    Q_UNUSED(env);
    Q_UNUSED(thiz);

    AudioBridge *bridge = AudioBridge::instance();
    if (bridge) {
        QMetaObject::invokeMethod(bridge, [bridge, position, duration]() {
            bridge->handlePositionChangedFromJava(position, duration);
        }, Qt::QueuedConnection);
    }
}
#endif
```

Figura 33 – AudioBridge

D. O Desacoplamento na Interface (QML)

Do lado do ecrã (TourPage.qml), a interface foi projetada para ser completamente "ignorante" sobre o leitor de áudio real. O QML apenas envia comandos simples (AudioBridge.startAudio() ou AudioBridge.stopAudio()) e reage de forma passiva à informação que chega do C++.

- **Sincronização Visual:** As propriedades fakePosition e fakeDuration recebem constantemente os valores filtrados do C++. A barra dourada do ecrã baseia o seu preenchimento e largura no cálculo (fakePosition / fakeDuration) * parent.width, movendo-se autonomamente e sem interrupções de hardware.
- **Controlo Manual Inteligente:** Quando o utilizador toca e arrasta a barra de progresso (MouseArea com drag.target), o QML suspende temporariamente a atualização visual e, ao soltar o dedo, recalcula a percentagem do ecrã e avisa imediatamente o C++: *"O utilizador quer ouvir o áudio a partir do segundo X"*. O C++ envia essa ordem para o Java e o áudio é ajustado instantaneamente.

Esta separação clara de responsabilidades resulta numa aplicação extremamente robusta: a interface permanece leve e fluida, a estabilidade é garantida pelo C++ e a reprodução de áudio é mantida viva e indestrutível pelo sistema Android.

4.8.3 Identificação Inteligente e Tratamento de Itens Globais

A experiência de visitar um museu é, por natureza, orgânica e não linear. Embora a aplicação ofereça uma rota planeada (sequencial) para guiar o utilizador de forma lógica pelas salas, o comportamento real de um visitante envolve desvios: o utilizador pode estar a seguir o circuito de áudio-guia em segundo plano, mas sentir-se subitamente atraído por uma obra de arte exposta num corredor secundário que não faz parte daquela visita guiada específica.

Para responder a esta imprevisibilidade sem gerar fricção tecnológica ou obrigar o utilizador a abandonar o seu percurso, foi desenhada uma lógica flexível de identificação de obras através de códigos QR ou PINs de 4 dígitos. Esta abordagem divide-se em dois comportamentos arquiteturais automatizados:

A. Redirecionamento Dinâmico Dentro do Circuito

Quando o utilizador decide interagir diretamente com uma peça usando o digitalizador ou o teclado virtual, o sistema faz uma validação prioritária em circuito fechado. Se o código introduzido corresponder a uma obra que pertence à visita guiada atual, a aplicação atua como um atalho inteligente:

Este fluxo está ilustrado na Figura 34, onde se observa o momento em que o utilizador efetua a leitura de um código QR válido e o sistema, de forma imediata, redireciona a interface e o motor de áudio nativo para a respetiva paragem cronológica dentro do circuito.

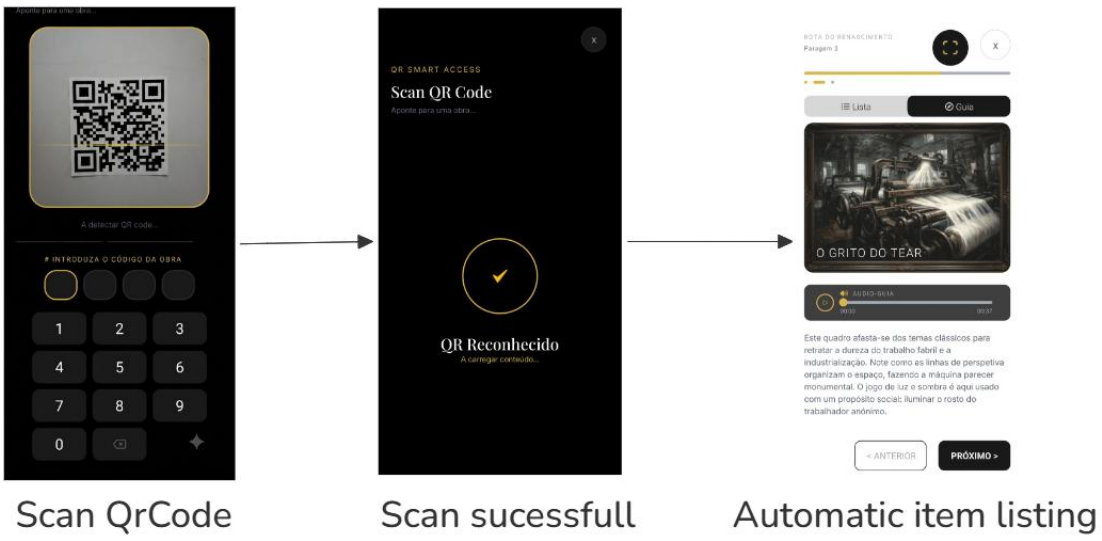


Figura 34 - Leitura dinamica de Item do Tour

B. Isolamento Adaptativo de Itens Globais (Fora do Percurso)

O maior desafio arquitetural surge quando o visitante digitaliza uma obra de arte totalmente exterior à rota ativa. Numa aplicação convencional, isto causaria um erro ou forçaria o encerramento da visita guiada atual para abrir a nova obra, destruindo o histórico do utilizador.

Para resolver este problema, a aplicação implementa um mecanismo de Isolamento de Estado (Fallback). Caso o código lido pertença ao acervo global do museu, mas não à rota em curso, a aplicação reage de forma isolada, como esquematizado na Figura 35.

1. **Preservação do Progresso:** O estado da visita guiada (a paragem atual, o progresso do circuito e as definições da rota) é imediatamente congelado e guardado em memória secundária, ficando completamente protegido contra alterações.
2. **Ativação da Camada Autónoma (Overlay):** O áudio do percurso principal é interrompido de forma segura e a aplicação sobrepõe um ecrã isolado (um painel flutuante global) que exibe as informações e reproduz o áudio da obra avulsa.
3. **Restauro Transparente:** O utilizador pode contemplar a obra extra pelo tempo que desejar. Quando decide fechar esse painel de visualização livre, o ecrã isolado é destruído, o sistema recupera instantaneamente o estado guardado e o visitante regressa exatamente ao ponto exato do circuito onde se encontrava, retomando o áudio original.

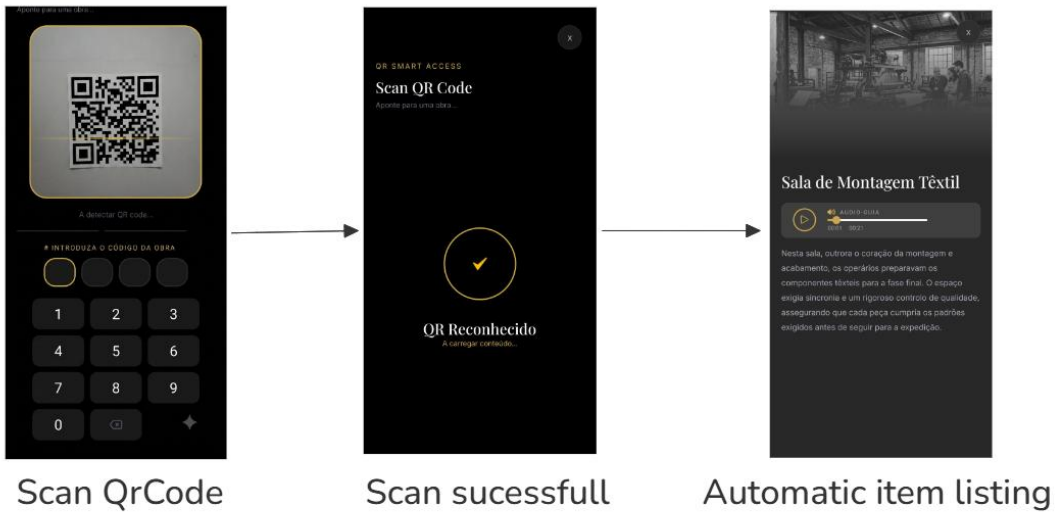


Figura 35 - Leitura dinâmica de Item fora do Tour

4.9 Assistente Virtual (ChatIAPanel.qml)

Durante uma visita a um museu, nem todas as dúvidas do visitante encontram resposta direta através das fichas dos itens ou das áreas disponíveis no mapa interativo. Questões relacionadas com a história da instituição, contexto de determinadas peças, funcionamento do espaço ou organização das visitas guiadas poderiam obrigar o visitante a abandonar o percurso atual e procurar manualmente a informação disponível.

4.9.1 Motivação e objetivo da funcionalidade

Para ultrapassar esta limitação, foi desenvolvido um assistente virtual baseado em inteligência artificial, permitindo ao visitante interagir com a aplicação através de linguagem natural. Esta funcionalidade transforma a aplicação de um simples catálogo digital num sistema de apoio à visita capaz de responder dinamicamente às necessidades do visitante.

Um dos principais objetivos do desenvolvimento foi garantir que a inteligência artificial complementa, mas não substitui, a informação oficial disponibilizada pelo museu. Por esse motivo, a solução foi construída seguindo uma abordagem catalog-first, onde o conteúdo existente no sistema de gestão Strapi representa sempre a fonte prioritária das respostas.

4.9.2 Integração do assistente na aplicação

O assistente virtual encontra-se disponível através de um botão flutuante presente em diferentes áreas da aplicação, nomeadamente na página inicial (HomePage) e durante a consulta de visitas guiadas.

Ao selecionar esta opção, é apresentado o componente ChatIAPanel.qml, responsável pela interface conversacional. Este componente permite ao visitante introduzir perguntas, visualizar o histórico da conversa durante a sessão e receber respostas sem abandonar o contexto da visita como podemos ver na figura 36.

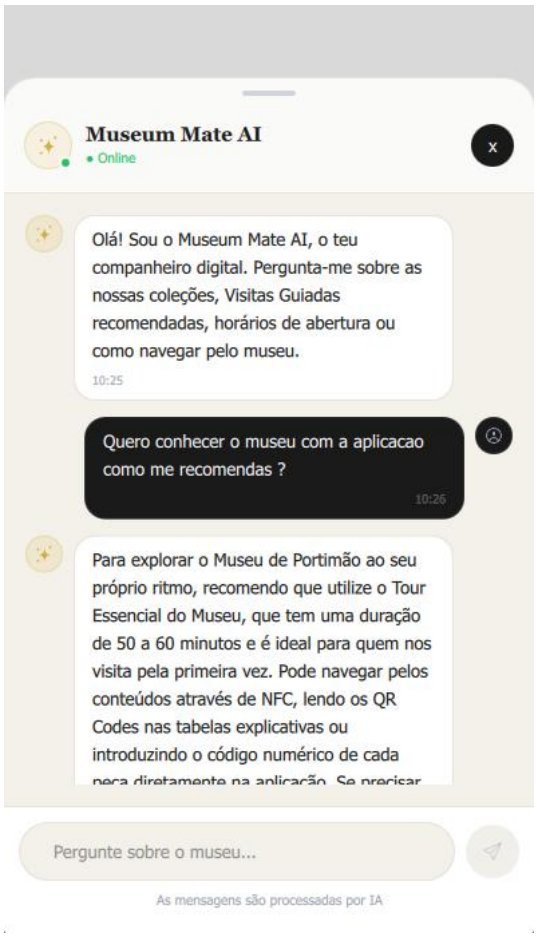


Figura 36 – Chat IA

A comunicação entre a aplicação móvel e o serviço de inteligência artificial é realizada através de pedidos HTTP para o backend desenvolvido em Strapi. Desta forma, a aplicação móvel mantém apenas a responsabilidade de apresentar a interface e enviar os pedidos, enquanto toda a lógica de processamento das perguntas permanece centralizada no servidor.

Esta separação permite alterar ou evoluir o comportamento do assistente sem necessidade de atualizar a aplicação instalada nos dispositivos dos visitantes.

4.9.3 Arquitetura e processamento das perguntas

O funcionamento do assistente foi desenvolvido sobre uma arquitetura composta por três etapas principais:

- **Consulta do catálogo oficial do museu**
- **Classificação da pergunta**
- **Geração da resposta final**

Quando o visitante envia uma pergunta, o backend começa por disponibilizar à inteligência artificial o contexto existente no catálogo Strapi, incluindo:

- informação das entidades do museu;
- áreas;
- itens;
- visitas guiadas;
- conteúdos multilingues existentes.

A primeira análise determina se a pergunta pode ser respondida utilizando exclusivamente a informação oficial disponível.

Cada pergunta é classificada numa de três categorias:

Respondível pelo catálogo:
Existe informação suficiente no conteúdo do museu, sendo utilizada exclusivamente essa informação.

Relacionada com o museu, mas sem informação suficiente no catálogo:
A pergunta mantém relação com a instituição, mas os conteúdos existentes não permitem gerar uma resposta completa. Neste caso é ativado um mecanismo de fallback baseado no conhecimento geral do modelo de IA.

Fora do domínio do museu:
A pergunta não apresenta relação com a visita ou com a instituição. Nestes casos, o assistente recusa responder, mantendo o comportamento de um guia especializado e evitando funcionar como um chatbot genérico.

Esta classificação é fundamental para preservar o propósito da aplicação, garantindo que a inteligência artificial é utilizada como ferramenta complementar à experiência museológica.

4.9.4 Sistema de fallback e controlo da informação gerada

Quando o catálogo não possui informação suficiente, mas a pergunta continua relacionada com o museu, o sistema pode recorrer a uma segunda chamada ao modelo de inteligência artificial utilizando conhecimento geral previamente adquirido pelo modelo.

Esta segunda etapa foi desenvolvida como um mecanismo de complemento e não como substituição do catálogo oficial. Assim, existem regras específicas para controlar a qualidade das respostas:

- a informação existente no catálogo tem sempre prioridade;
- respostas provenientes do conhecimento geral são identificadas visualmente ao visitante;
- quando não existe informação suficiente ou confiança na resposta, o assistente informa o visitante que não possui dados disponíveis em vez de gerar informação especulativa.

Esta abordagem reduz o risco de apresentar informação incorreta e mantém uma distinção clara entre:

- informação validada pelo museu;
- informação complementar gerada pela inteligência artificial.

Foi também implementado um mecanismo de configuração central (KNOWLEDGE_FALLBACK_ENABLED) que permite ativar ou desativar esta funcionalidade sem alterações no código da aplicação. Esta decisão permite controlar o comportamento do sistema consoante o contexto de utilização, custos associados ou necessidades específicas de cada instituição.

4.9.5 Suporte multilingue e adaptação das respostas

O assistente foi integrado com o sistema multilingue existente na aplicação, suportando os idiomas definidos para a experiência de visita.

Quando a informação necessária existe apenas num idioma diferente daquele selecionado pelo visitante, o sistema realiza automaticamente a tradução da resposta antes da apresentação.

Nestes casos, a aplicação identifica visualmente que a resposta foi traduzida automaticamente, permitindo distinguir conteúdos originalmente disponibilizados pelo museu daqueles que passaram por um processo de tradução automática.

A mesma lógica é aplicada às respostas provenientes do fallback de conhecimento geral, mantendo sempre visível quando a informação apresentada não pertence diretamente ao catálogo oficial.

4.9.6 Recolha de dados e melhoria contínua dos conteúdos

Todas as interações realizadas com o assistente são integradas no sistema de analytics existente na aplicação.

Sempre que o mecanismo de fallback é utilizado, é criado um evento específico (assistant_knowledge_fallback) contendo apenas informação técnica necessária para análise, como:

- idioma utilizado;
- data/hora;
- ocorrência do fallback.

As perguntas e respostas completas não são armazenadas, preservando a privacidade do visitante.

Estes dados permitem identificar lacunas existentes no catálogo do museu, funcionando como uma ferramenta de apoio à evolução futura dos conteúdos. Por exemplo, se determinados temas forem frequentemente questionados e necessitarem de recurso ao conhecimento externo, esses casos podem indicar novos conteúdos que devem ser adicionados ao sistema de gestão.

4.9.7 Impacto da funcionalidade

A implementação do assistente virtual acrescenta uma nova camada de interação à visita museológica, permitindo ao visitante obter esclarecimentos de forma imediata sem interromper o percurso.

Do ponto de vista técnico, a solução demonstra a integração entre uma aplicação móvel desenvolvida em Qt/QML, um backend Strapi e serviços de inteligência artificial, mantendo princípios importantes como controlo da fonte de informação, separação de responsabilidades e possibilidade de evolução futura.

Do ponto de vista do museu, além de melhorar a experiência do visitante, a funcionalidade fornece indicadores sobre dúvidas frequentes e possíveis lacunas informativas, contribuindo para uma melhoria contínua do conteúdo disponibilizado.

5. Conclusão

Em conclusão, o estágio curricular constituiu uma etapa determinante na minha formação enquanto futuro profissional da área do desenvolvimento de software. Ao longo deste período tive a oportunidade de aplicar os conhecimentos adquiridos durante o curso e, sobretudo, desenvolver novas competências técnicas num contexto profissional real.

O percurso realizado demonstrou uma evolução gradual, iniciando-se com a aprendizagem da framework Qt e da linguagem QML através de pequenos projetos práticos, que permitiram consolidar conceitos fundamentais de programação, interfaces gráficas e organização do código. Posteriormente, essa base revelou-se essencial para o desenvolvimento do projeto Museum Mate, um sistema de maior dimensão que exigiu uma abordagem muito mais estruturada ao nível da arquitetura de software, levantamento de requisitos, planeamento, modelação de dados e integração de diferentes tecnologias.

Para além do desenvolvimento da aplicação móvel, o estágio permitiu aprofundar conhecimentos em arquiteturas Headless CMS, Docker, desenvolvimento backend, sistemas offline-first, internacionalização, serviços nativos Android, integração de inteligência artificial e metodologias modernas de desenvolvimento orientadas por especificações. Estas experiências contribuíram para compreender a importância de tomar decisões arquiteturais antes da implementação, privilegiando soluções escaláveis, reutilizáveis e de fácil manutenção.

A nível pessoal, este estágio permitiu desenvolver autonomia, capacidade de análise crítica, resolução de problemas e adaptação a novos desafios tecnológicos, competências fundamentais para qualquer profissional da engenharia de software.

Considero que os objetivos inicialmente definidos foram plenamente alcançados e que esta experiência representou uma excelente preparação para a minha futura integração no mercado de trabalho, proporcionando uma visão muito mais completa do processo de desenvolvimento de software em ambiente empresarial.

6. Bibliografia

Android Developers. (s.d.). *Foreground services overview*. Consultado em https://developer.android.com/develop/background-work/services/foreground-services

Anthropic. (s.d.). *Claude Code – Documentação*. Consultado em https://docs.claude.com/en/docs/claude-code/overview

Docker Inc. (s.d.). *Docker Documentation*. Consultado em https://docs.docker.com/

EDIGMA. (s.d.). *EDIGMA – Experiências para pessoas*. Consultado em https://edigma.com/

EDIGMA. (s.d.). *Digital Signage – Sinalética Digital*. Consultado em https://edigma.com/sinaletica-digital-portugal/

EDIGMA. (2023). *Fundação de Serralves – Rede de Sinalética Digital*. Consultado em https://edigma.com/fundacaoserralves/

EDIGMA. (s.d.). *Contactos*. Consultado em https://edigma.com/contactos/

Git. (s.d.). *Git Documentation*. Consultado em https://git-scm.com/doc

GitHub. (s.d.). *Spec Kit*. Consultado em https://github.com/github/spec-kit

LinkedIn. (s.d.). *EDIGMA*. Consultado em https://pt.linkedin.com/company/edigma

Marketeer. (2025, 31 de outubro). *Daniel Afonso, CMO da EDIGMA é o primeiro português distinguido na lista "Inavate EMEA 40 Under 40"*. Consultado em https://marketeer.sapo.pt/daniel-afonso-cmo-da-edigma-e-o-primeiro-portugues-distinguido-na-lista-inavate-emea-40-under-40/

Meta Platforms Inc. (s.d.). *React Documentation*. Consultado em https://react.dev/

O Minho. (2021). *Empresa de Braga cria tecnologia para garantir distanciamento social em filas*. Consultado em https://ominho.pt/empresa-de-braga-cria-tecnologia-para-garantir-distanciamento-social-em-filas/

Qt Group. (s.d.). *Qt Documentation*. Consultado em https://doc.qt.io/

Qt Group. (s.d.). *Qt Quick and QML Documentation*. Consultado em https://doc.qt.io/qt-6/qtquick-index.html

Qt Group. (s.d.). *Qt for Android*. Consultado em https://doc.qt.io/qt-6/android.html

SQLite. (s.d.). *SQLite Documentation*. Consultado em https://www.sqlite.org/docs.html

Strapi Solutions. (s.d.). *Strapi Documentation*. Consultado em https://docs.strapi.io/

Strapi Solutions. (s.d.). *Strapi REST API*. Consultado em https://docs.strapi.io/dev-docs/api/rest

ZXing Project. (s.d.). *ZXing ("Zebra Crossing") *. Consultado em https://github.com/zxing/zxing